Appendix

A.1 Vectors and Matrices

A.1.1 Notation and Definitions

A single number is often called a *scalar* and is represented by italics, for example, x.

A vector consists of a row or column of numbers and is represented by bold lower case italics, for example, $\boldsymbol{x}$. For example, $\mathbf{x} = \begin{bmatrix} 3 & -11 & 9 & 0 \end{bmatrix}$ is a row vector and $\boldsymbol{y} = \begin{bmatrix} 5.6 \\ 2.8 \\ 1.9 \end{bmatrix}$ is a column vector.

A matrix is a two-dimensional array of numbers and is represented by bold upper case italics, for example, $\boldsymbol{X}$. For example, $\boldsymbol{X} = \begin{bmatrix} 12 & 3 & 8 \\ -2 & 14 & 1 \end{bmatrix}$ is a matrix.

The dimensions of a matrix are normally presented with the number of rows first and the number of columns second, and vectors can be considered as matrices with one dimension equal to 1, so that $\boldsymbol{x}$ above has dimensions 1×4 and $\boldsymbol{X}$ has dimensions 2×3.

A square matrix is one where the number of columns equals the number of rows. For example, $\boldsymbol{Y} = \begin{bmatrix} -7 & 4 & -1 \\ 11 & -3 & 6 \\ 2 & 4 & -12 \end{bmatrix}$ is a square matrix.

An identity matrix is a square matrix whose elements are equal to 1 in the diagonal and 0 elsewhere and is often denoted by $\boldsymbol{I}$. For example, $\boldsymbol{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ is an identity matrix.

Each number in a matrix is called an *element* and is often referenced as a scalar, with subscripts referring to the row and column; hence, in the matrix above, $y_{21} = 11$, which is the element in row 2 and column 1. Optionally, a comma can be placed between the subscripts for clarity; this is useful if one of the dimensions exceeds 9.

A.1.2 Matrix and Vector Operations

A.1.2.1 Addition and Subtraction

Addition and subtraction are the most straightforward operations. Each matrix (or vector) must have the same dimensions and simply involves performing the operation element by element. Hence,

$$\begin{bmatrix} 9 & 7 \\ -8 & 4 \\ -2 & 4 \end{bmatrix} + \begin{bmatrix} 0 & -7 \\ -11 & 3 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} 9 & 0 \\ -19 & 7 \\ 3 & 10 \end{bmatrix}$$

A.1.2.2 Transpose

Transposing a matrix involves swapping the columns and rows around and may be denoted by a right-hand side superscript ‘′’. For example, if $\boldsymbol{Z} = \begin{bmatrix} 3.1 & 0.2 & 6.1 & 4.8 \\ 9.2 & 3.8 & 2.0 & 5.1 \end{bmatrix}$, then $\boldsymbol{Z}' = \begin{bmatrix} 3.1 & 9.2 \\ 0.2 & 3.8 \\ 6.1 & 2.0 \\ 4.8 & 5.1 \end{bmatrix}$. Some authors used a superscripted ‘T’ instead.

Chemometrics: Data Driven Extraction for Science, Second Edition. Richard G. Brereton.
© 2018 John Wiley & Sons Ltd. Published 2018 by John Wiley & Sons Ltd.
Companion website: http://booksupport.wiley.com

A.1.2.3 Multiplication

Matrix and vector multiplication using the 'dot' product is sometimes denoted by the symbol '·' between matrices, to distinguish from the cross product sometimes encountered in physics but rarely in chemometrics. However, authors often omit the '·' as we will do in this text.

It is only possible to multiply two matrices together if the number of columns of the first matrix equals the number of rows of the second matrix. The number of rows of the product will equal the number of rows of the first matrix, and the number of columns equals the number of columns of the second matrix. Hence, a 3×2 matrix when multiplied by a 2×4 matrix will give a 3×4 matrix.

Multiplication of matrices is not commutative, that is generally $\boldsymbol{AB} \neq \boldsymbol{BA}$ even if the second product is allowed. Matrix multiplication can be expressed in the form of summations. For arrays with more than two dimensions (e.g. tensors), conventional symbolism can be quite awkward and it is probably easier to think in terms of summations.

If matrix $\boldsymbol{A}$ has dimensions $I \times J$ and matrix $\boldsymbol{B}$ has dimensions $J \times K$, then the product $\boldsymbol{C}$ of dimensions $I \times K$ has elements defined by

$$c_{ik} = \sum_{j=1}^{J} a_{ij} b_{jk}$$

Hence,

$$\begin{bmatrix} 1 & 7 \\ 9 & 3 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 6 & 10 & 11 & 3 \\ 0 & 1 & 8 & 5 \end{bmatrix} = \begin{bmatrix} 6 & 17 & 67 & 38 \\ 54 & 93 & 123 & 42 \\ 12 & 25 & 62 & 31 \end{bmatrix}$$

To illustrate this, the element of the first row and second column of the product is given by

$$17 = 1 \times 10 + 7 \times 1$$

When several matrices are multiplied together, it is normal to take any two neighbouring matrices, multiply them together and then multiply this product with another neighbouring matrix. It does not matter what order this is done in; hence, $\boldsymbol{ABC} = (\boldsymbol{AB})\boldsymbol{C} = \boldsymbol{A}(\boldsymbol{BC})$. Hence, matrix multiplication is associative. Matrix multiplication is also distributive, that is $\boldsymbol{A}(\boldsymbol{B} + \boldsymbol{C}) = \boldsymbol{AB} + \boldsymbol{AC}$.

A.1.2.4 Inverse

Most square matrices have inverses, defined by the matrix which when multiplied with the original matrix gives the identity matrix and is represented by a superscript '−1' as a right-hand side superscript, so that $\boldsymbol{DD}^{-1} = \boldsymbol{I}$. Note that some square matrices do not have inverses: this is caused by there being correlations in the original matrix which results in the determinant being zero: such matrices are called *singular* matrices.

A.1.2.5 Pseudo-Inverse

In several sections of this text, we use the idea of a pseudo-inverse. If matrices are not square, it is not possible to calculate an inverse, but the analogous concept of a pseudo-inverse exists and is employed in regression analysis.

If $\boldsymbol{A} = \boldsymbol{BC}$ then $\boldsymbol{B}'\boldsymbol{A} = \boldsymbol{B}'\boldsymbol{BC}$ so $(\boldsymbol{B}'\boldsymbol{B})^{-1}\boldsymbol{B}'\boldsymbol{A} = \boldsymbol{C}$ and $(\boldsymbol{B}'\boldsymbol{B})^{-1}\boldsymbol{B}'$ is said to be the left pseudo-inverse of $\boldsymbol{B}$.

Equivalently, $\boldsymbol{AC}' = \boldsymbol{BCC}'$ so $\boldsymbol{AC}'(\boldsymbol{CC}')^{-1} = \boldsymbol{B}$ and $\boldsymbol{C}'\ (\boldsymbol{CC}')^{-1}$ is said to be the right pseudo-inverse of $\boldsymbol{C}$.

Sometimes, the pseudo-inverse is denoted by a superscript '+'. Hence, $\boldsymbol{B}^{+}$ is the pseudo-inverse of $\boldsymbol{B}$. There is not usually a distinction in notation between right and left, but the reader should be able to work this out from context. Note that if a matrix is not square (in which case there is no need to calculate a pseudo-inverse), only one of the two possible pseudo-inverses will exist: if the number of rows is less than the columns, it is the right pseudo-inverse, otherwise the left one.

In regression, the equation $\boldsymbol{A} \approx \boldsymbol{BC}$ is an approximation; for example, $\boldsymbol{A}$ may represent a series of spectra that are approximately equal to the product of two matrices such as scores and loadings matrices, and the pseudo-inverse can be used to obtain the best fit model (or estimate) of $\boldsymbol{C}$ knowing $\boldsymbol{A}$ and $\boldsymbol{B}$ or of $\boldsymbol{B}$ knowing $\boldsymbol{A}$ and $\boldsymbol{C}$.

A.1.2.6 Trace and Determinant

Other properties of square matrices sometimes encountered are the trace, which is the sum of the diagonal elements and the determinant which relates to the size of the matrix. A determinant of 0 indicates a matrix without an inverse. A determinant close to 0 often suggests that the data are quite correlated or a poor experimental design resulting in fairly unreliable predictions or that there are many redundant noisy variables. If the dimensions of matrices are large

and the magnitudes of the elements are small (e.g. 10^{-3}), it is sometimes possible to obtain a determinant close to zero even though the matrix has an inverse; a solution to this problem is to multiply each element by a number such as 10^3 and then remember to readjust the magnitude of the numbers in resultant computation to take into account of this later.

A.1.2.7 Vector Magnitude

An interesting property chemometricians sometimes use is that the product of the transpose of a column vector with itself equals the sum of square of elements of the vector, so that $\boldsymbol{x}'\boldsymbol{x} = \Sigma x^2$. The magnitude, or length, of a vector is given by $\sqrt{(\boldsymbol{x}'\boldsymbol{x})} = \sqrt{\sum x^2}$ or the square root of the sum of its elements. This can be visualised in geometry as the length of the line from the origin to the point in space indicated by the vector co-ordinates. Note that there is some confusion in terminology, as the length of a vector can also denote the number of elements in a vector, so the term magnitude is less ambiguous.

A.2 Algorithms

There are many different descriptions of the various algorithms in the literature. This Appendix describes one algorithm for each of four regression methods.

A.2.1 Principal Components Analysis

NIPALS is a common, iterative algorithm often used for principal components analysis (PCA). Some authors use another method called *singular value decomposition* (SVD). The main difference is that NIPALS extracts components one at a time and can be stopped after the desired number of PCs is obtained. In the case of large data sets with, for example, 200 variables (e.g. in spectroscopy), this can be very useful and reduce the amount of effort required. In this text, we will restrict to NIPALS for simplicity.

The steps are as follows.

Initialisation

1. Take a matrix $\boldsymbol{Z}$ and if required pre-process (e.g. mean centre or standardise) to give the matrix $\boldsymbol{X}$, which is used for PCA.

New Principal Component

2. Take a column of this matrix (often the column with greatest sum of squares) as the first guess of the scores first principal component, call it ${}^{initial}\hat{\boldsymbol{t}}$.

Iteration for Each Principal Component

3. Calculate

$$ {}^{unnorm}\hat{\boldsymbol{p}} = \frac{{}^{initial}\hat{\boldsymbol{t}}'\mathbf{X}}{\sum \hat{t}^2} $$

4. Normalise the guess of the loadings, so

$$ \hat{\boldsymbol{p}} = \frac{{}^{unnorm}\hat{\boldsymbol{p}}}{\sqrt{\sum {}^{unnorm}\hat{p}^2}} $$

5. Now calculate a new guess of the scores

$$ {}^{new}\hat{\boldsymbol{t}} = \boldsymbol{X}\hat{\boldsymbol{p}}' $$

Check for Convergence

6. Check if this new guess differs from the first guess, a simple approach is to look at the size of the sum of square difference in the old and new scores, that is, $\sum({}^{initial}\hat{t} - {}^{new}\hat{t})^2$. If this is small, the PC has been extracted, set the PC scores ($\boldsymbol{t}$) and loadings ($\boldsymbol{p}$) for the current PC to $\hat{\boldsymbol{t}}$ and $\hat{\boldsymbol{p}}$. Otherwise, return to step 3, substituting the initial scores by the new scores.

Compute the Component and Calculate Residuals

7. Subtract the effect of the new PC from the data matrix to get a residual data matrix

$$^{resid}\boldsymbol{X} = \boldsymbol{X} - \boldsymbol{tp}$$

Further PCs

8. If it is desired to compute further PCs, substitute the residual data matrix for $\boldsymbol{X}$ and go to step 2.

The scores of a new sample that was not part of the original training set (e.g. test set) can be estimated by

$$\hat{\boldsymbol{t}} = \boldsymbol{xP'}$$

where $\hat{\boldsymbol{t}}$ is a row vector of dimensions $1 \times A$ with A the number of components.

A.2.2 PLS1

There are several implementations, the one below is non-iterative.

Initialisation

1. Take a matrix $I \times J$ and if required pre-process (e.g. mean centre or standardise) to give the matrix $\boldsymbol{X}$ which is used for PLS.
2. Take the concentration (or c) vector and pre-process it to give the vector $\boldsymbol{c}$, which is used for PLS. Note that if the data matrix $\boldsymbol{Z}$ is centred down the columns, $\boldsymbol{c}$ must also be centred. Generally, centring is the only form of pre-processing useful for PLS1. Start with an estimate of $\hat{\boldsymbol{c}}$ that is a vector of 0s (equal to its mean if the vector is already centred).

New PLS Component

3. Calculate the $J \times 1$ vector $\boldsymbol{h}$

$$\boldsymbol{h} = \boldsymbol{X'c}$$

4. Normalise this vector to give $\boldsymbol{w}$, which is called the *weights vector*

$$\boldsymbol{w} = \frac{\boldsymbol{h}}{\sqrt{\sum h^2}}$$

5. Calculate the $I \times 1$ scores vector, which is simply given by

$$\boldsymbol{t} = \boldsymbol{Xw}$$

6. Calculate the $1 \times Jx$ loadings by

$$\boldsymbol{p} = \frac{\boldsymbol{t'X}}{\sum t^2}$$

7. Calculate the c loading (a scalar) by

$$q = \frac{\boldsymbol{c't}}{\sum t^2}$$

Compute the Component and Calculate Residuals

8. Subtract the effect of the new PLS component from the data matrix to get a residual data matrix

$$^{resid}\boldsymbol{X} = \boldsymbol{X} - \boldsymbol{tp}$$

9. Determine the new estimate of $\boldsymbol{c}$ by

$$^{new}\hat{\boldsymbol{c}} = {}^{initial}\hat{\boldsymbol{c}} + \boldsymbol{t}q$$

and sum the contribution of all components calculated to give an estimated $\hat{\boldsymbol{c}}$. Note that the initial estimate is 0 (or the mean) before the first component has been computed. Calculate

$$^{resid}\boldsymbol{c} = {}^{true}\boldsymbol{c} - {}^{new}\hat{\boldsymbol{c}}$$

where $^{true}\boldsymbol{c}$ is, like all values of c, after the data have been pre-processed (such as centring).

Further PLS Components

10. If further components are required, replace both $\boldsymbol{X}$ and $\boldsymbol{c}$ by the residuals and return to step 3.

Note that in the implementation used in this text, the PLS loadings are neither normalised nor orthogonal. However, the weights vectors are both orthogonal and normal. Remember to add back the mean when comparing the estimates with the observed data.

There are several different PLS1 algorithms; hence, it is useful to check exactly what method a particular package uses, although the resultant concentration estimates should be identical for each method (unless there is a problem with convergence in iterative approaches).

A separate step involves prediction of test set or unknown samples. Unlike PCA, PLS does not result in a best least squares fit to the original $\boldsymbol{X}$ matrix, so prediction is more complicated than for PCA. There are several ways of expressing this, of which the following is an easy one for A components.

1. Calculate $\boldsymbol{b} = \boldsymbol{W}\ (\boldsymbol{PW})^{-1}\boldsymbol{q}$, where $\boldsymbol{W}$ is the $J \times A$ weights matrix, each column representing a component. $\boldsymbol{P}$ the $A \times J$ x loadings matrix, and $\boldsymbol{q}$ an $A \times 1$ vector of the $\boldsymbol{c}$ loadings.
2. As appropriate, calculate $b_0 = \bar{c} - \bar{\boldsymbol{x}}b$ if it is required to transform the predictions to the original data.
3. Estimate $\hat{c} = b_0 + \boldsymbol{x}\boldsymbol{b}$.

Similar estimations could be made for the x block but the reader is referred to the specialist literature.

A.2.3 PLS2

This is a straightforward, iterative extension of PLS1. Only small variations are required. Instead of $\boldsymbol{c}$ being a vector, it is now a $I \times$ N matrix $\boldsymbol{C}$ and instead of q being a scalar, it is now a $1 \times N$ vector $\boldsymbol{q}$, where there are N columns in the $\boldsymbol{c}$ matrix (e.g. concentrations of analytes).

Initialisation

1. Take the data matrix and if required pre-process (e.g. mean centre or standardise) to give the matrix $\boldsymbol{X}$, which is used for PLS.
2. Take the concentration matrix and pre-process it to give the matrix $\boldsymbol{C}$, which is used for PLS. Note that if the data matrix is centred down the columns, the vector matrix must also be centred. Generally, centring is the only form of pre-processing useful for PLS2. Start with an estimate of $\hat{\boldsymbol{C}}$ that is a matrix of 0s (equal to the mean value of c if already centred).

New PLS Component

3. An extra step is required to identify a vector $\boldsymbol{u}$, which can be a guess (as in PCA), but can be chosen as one of the columns in the initial pre-processed concentration matrix, $\boldsymbol{C}$.
4. Calculate the vector
$$\boldsymbol{h} = \boldsymbol{X}'\boldsymbol{u}$$
5. Normalise to get the weights by
$$\boldsymbol{w} = \frac{\boldsymbol{h}}{\sqrt{\sum \boldsymbol{h}^2}}$$
6. Calculate the guessed scores by
$$^{new}\hat{\boldsymbol{t}} = \boldsymbol{X}\boldsymbol{w}$$
7. Calculate the guessed x loadings by
$$\hat{\boldsymbol{p}} = \frac{\hat{\boldsymbol{t}}'\boldsymbol{X}}{\sum \hat{t}^2}$$
8. Calculate the c loadings (a $1 \times N$ vector rather than scalar in PLS2) by
$$\hat{\boldsymbol{q}} = \frac{\boldsymbol{C}'\hat{\boldsymbol{t}}}{\sum \hat{t}^2}$$
9. If this is the first iteration, remember the scores and call them $^{initial}\boldsymbol{t}$, then produce a new vector $\boldsymbol{u}$ by
$$\boldsymbol{u} = \frac{\boldsymbol{C}\hat{\boldsymbol{q}}}{\sum q^2}$$
and return to step 4.

Check for Convergence

10. If this is the second time, round compare the new and old scores vectors, for example, by looking at the size of the sum of square difference in the old and new scores, that is, $\sum({}^{initial}\hat{t} - {}^{new}\hat{t})^2$. If this is small, the PLS component has been adequately modelled, set the PLS scores ($\boldsymbol{t}$) and both types of loadings ($\boldsymbol{p}$ and $\boldsymbol{c}$) for the current PC to $\hat{\boldsymbol{t}}$ and $\hat{\boldsymbol{p}}$, and $\hat{\boldsymbol{q}}$. Otherwise, calculate a new value of $\boldsymbol{u}$ as in step 8 and return to step 4.

Compute the Component and Calculate the Residuals

11. Subtract the effect of the new PLS component from the data matrix to get a residual data matrix

$$^{resid}\boldsymbol{X} = \boldsymbol{X} - \boldsymbol{tp}$$

12. Determine the new $\boldsymbol{c}$ matrix estimate by

$$^{new}\hat{\boldsymbol{C}} = {}^{initial}\hat{\boldsymbol{C}} + \boldsymbol{tq}$$

and sum the contribution of all components calculated to give an estimated $\hat{\boldsymbol{c}}$. Calculate

$$^{resid}\boldsymbol{C} = {}^{true}\boldsymbol{C} - \hat{\boldsymbol{C}}$$

Further PLS Components

13. If further components are required, replace both $\boldsymbol{X}$ and $\boldsymbol{C}$ by the residuals and return to step 3.

Prediction of new samples that are not part of the training set involves similar principles to PLS1, except we are predicting a $\boldsymbol{C}$ matrix rather than vector.

A.2.4 Tri-Linear PLS1

The algorithm below is closely based on PLS1 and is suitable when there is only one column in the $\boldsymbol{c}$ vector.

Initialisation

1. Take a three-way tensor $\underline{\boldsymbol{Z}}$ and if required pre-process (e.g. mean centre or standardise) to give the tensor $\underline{\boldsymbol{X}}$, which is used for PLS. Perform all pre-processing on this tensor. The tensor has dimensions $I \times J \times K$.
2. Pre-process the concentrations if appropriate to give a vector $\boldsymbol{c}$.

New PLS Component

3. From the original tensor, create a new matrix $\boldsymbol{H}$ with dimensions $J \times K$, which is the sum of each of the I matrices for each of the samples multiplied by the concentration of the analyte for the relevant sample, that is,

$$\boldsymbol{H} = \boldsymbol{X}_1 c_1 + \boldsymbol{X}_2 c_2 + \cdots + \boldsymbol{X}_I c_I$$

or, as a summation

$$h_{jk} = \sum_{i=1}^{I} c_i x_{ijk}$$

4. Perform PCA on $\boldsymbol{H}$ to obtain the scores and loadings, ${}^{h}\boldsymbol{t}$ and ${}^{h}\boldsymbol{p}$, for the first PC of $\boldsymbol{H}$. Note that only the first PC is retained, and for each new PLS component, a fresh $\boldsymbol{H}$ matrix is obtained.
5. Calculate the two x loadings for the current PLS component of the overall data set by normalising the scores and loadings of $\boldsymbol{H}$, that is,

$$^{j}\boldsymbol{p} = \frac{^{h}\boldsymbol{t'}}{\sqrt{\sum {}^{h}t^2}}$$

$$^{k}\boldsymbol{p} = \frac{^{h}\boldsymbol{p}}{\sqrt{\sum {}^{h}p^2}}$$

(This step is generally not necessary for most PCA algorithms.)

6. Calculate the overall scores by

$$t_i = \sum_{j=1}^{J}\sum_{k=1}^{K} x_{ijk}\, {}^{j}p_j\, {}^{k}p_k$$

7. Calculate the $\boldsymbol{c}$ loadings vector

$$\boldsymbol{q} = (\boldsymbol{T}'\boldsymbol{T})^{-1}\boldsymbol{T}'\boldsymbol{c}$$

where $\boldsymbol{T}$ is the scores matrix each column consisting of one component (a vector for the first PLS component).

Compute the Component and Calculate Residuals

8. Subtract the effect of the new PLS component from the original data matrix to get a residual data matrix (for each sample i)

$${}^{resid}\boldsymbol{X}_i = \boldsymbol{X}_i - t_i^j \boldsymbol{p}^k \boldsymbol{p}$$

9. Determine the new concentration estimates by

$$\hat{\boldsymbol{c}} = \boldsymbol{T}\boldsymbol{q}$$

Calculate

$${}^{resid}\boldsymbol{c} = {}^{true}\boldsymbol{c} - \hat{\boldsymbol{c}}$$

Further PLS Components

10. If further components are required, replace both $\boldsymbol{X}$ and $\boldsymbol{c}$ by the residuals and return to step 3.

A.3 Basic Statistical Concepts

There are numerous texts on basic statistics, some of them oriented towards chemists. It is not the aim of this section to provide a comprehensive background, but simply to provide the main definitions and tables that are helpful for using this text.

A.3.1 Descriptive Statistics

A.3.1.1 Mean

The mean of a series of measurements is defined by

$$\bar{x} = \sum_{i=1}^{I} x_i / I$$

Conventionally, a bar is placed above the letter. Sometimes, the letter m is used, but in this text, we will avoid this, as m is often used to denote an index. Hence, the mean of the measurement

4 8 5 −6 2 −5 6 0

$$\bar{x} = (4 + 8 + 5 - 6 + 2 - 5 + 6 + 0)/8 = 1.75$$

Statistically, this sample mean is often considered an estimate of the true population mean, sometimes denoted by μ. The population involves all possible samples of a given provenance, whereas only a selection is observed. In some cases, in chemometrics, this distinction is not so clear; for example, the mean intensity at a given wavelength over a chromatogram is an experimentally observed variable and it is not clear whether this mean is an estimate of all possible spectra of a given provenance or just something the experimenter obtained.

A.3.1.2 Variance and Standard Deviation

The estimated or sample variance of a series of measurements is defined by

$$v = \sum_{i=1}^{I} (x_i - \bar{x})^2 / (I - 1)$$

which can also be calculated using the equation

$$v = \sum_{i=1}^{I} x_i^2 / (I - 1) - \bar{x}^2 \times I / (I - 1)$$

So the estimated (sample) variance of the data in Section A.3.1.1 is

$$v = (4^2 + 8^2 + 5^2 + 6^2 + 2^2 + 5^2 + 6^2 + 0^2)/7 - 1.75^2 \times 8/7 = 25.928$$

This above equation is useful when it is required to estimate the population variance from a sample. However, the population variance is defined by

$$v = \sum_{i=1}^{I} (x_i - \bar{x})^2/I = \sum_{i=1}^{I} x_i^2/I - \bar{x}^2$$

One reason why there is a factor of $I - 1$ when using measurements in a number of samples to estimate population statistics is because one degree of freedom is lost when determining variance experimentally. For example, if we record one sample, the sum of squares $\sum_{i=1}^{I} (x_i - \bar{x})^2$ must be equal to 0, but this does not imply that the variance of the parent population is 0. As the number of samples increases, this small correction is not very important, and sometimes ignored. Statisticians might say that if we take, for example, 100 people from a town and calculate the variance of their heights, we are in practice estimating the variance of all people in the town and using our 100 measurements as a sample of maybe the full population of 1000. However, after we measure more than 10 heights, for example, it is unlikely that the difference between the population and sample means can be accurately estimated, indeed sampling would have to be very representative to obtain a detectable and reproducible difference.

The standard deviation, s, is simply the square root of the variance. The population standard deviation is sometimes denoted by σ.

In chemometrics, it is usual to use the population and not the sample standard deviation for standardising a data matrix. The reason is that we are not trying to estimate parameters in this case, but just to put different variables on a similar scale, when using Excel or Matlab, please ensure you use the right type.

A.3.1.3 Covariance and Correlation Coefficient

The covariance between two variables is a method for determining how closely they follow similar trends. It will never exceed in magnitude the geometric mean of the variance of the two variables, the lower the value, the less close the trends. Both variables must be measured on an identical number of samples, I in this case. The sample or estimated covariance between variables x and y is defined by

$$\text{cov}_{xy} = \sum_{i=1}^{I} (x_i - \bar{x})(y_i - \bar{y})/(I - 1)$$

whereas the population statistic is given by

$$\text{cov}_{xy} = \sum_{i=1}^{I} (x_i - \bar{x})(y_i - \bar{y})/I$$

Unlike the variance, it is perfectly possible for a covariance to take on negative values.

Many chemometricians prefer to use the Pearson's correlation coefficient, given by

$$r_{xy} = \frac{\text{cov}_{xy}}{s_x \cdot s_y} = \frac{\sum_{i=1}^{I} (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{I} (x_i - \bar{x})^2 \sum_{i=1}^{I} (y_i - \bar{y})^2}}$$

Notice that the definition of the correlation coefficient is identical both for samples and populations.

The correlation coefficient has a value between −1 and +1. If close to +1, the two variables are perfectly correlated. In many applications, correlation coefficients of −1 also indicate a perfect relationship. Under such circumstances, the value of y can be exactly predicted if we know x. The closer the correlation coefficients are to 0, the harder it is to use one variable to predict another. Some people prefer to use the square of the correlation coefficient, which varies between 0 and 1.

If two columns of a matrix have a correlation coefficient of ±1, the matrix is said to be rank deficient and has a determinant of 0, and so no inverse, this has consequences both in experimental design and in regression. There are various ways around this such as by removing selected variables or performing PCA.

In some areas of chemometrics, we used a *variance–covariance* matrix. This is a square matrix, whose dimensions usually equal the number of variables in a data set; for example, if there are 20 variables, the matrix has dimensions

20×20. The diagonal elements equal the variance of each variable and the off-diagonal elements the covariances. This matrix is symmetric about the diagonal. It is usual to employ population rather than sample statistics for this calculation as discussed above.

A.3.2 Normal Distribution

The normal distribution is an important statistical concept. There are many ways of introducing such distributions. Many texts use a probability density function

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

This rather complicated equation can be interpreted as follows. The function $f(x)$ is proportional to the probability that a measurement has a value x for a normally distributed population of mean μ and standard deviation σ. The function is scaled so that the area under the normal distribution curve is 1.

Most tables deal with the *standardised* normal distribution. This involves first standardising the raw data to give a new value z, the equation simplifies to

$$f(z) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{z^2}{2}\right)$$

Instead of calculating $f(z)$, most people look at the area under the normal distribution curve. This is proportional to the probability that a measurement is between certain limits. For example, the probability that a measurement is between 1 and 2 standard deviations can be calculated by taking the proportion of the overall area for which $1 \leq z \leq 2$.

These numbers can be obtained using simple functions, for example, in a spreadsheet, but are often conventionally presented in a tabular form. There are a surprisingly large number of types of tables, but Table A.1 allows the reader to calculate relevant information. This table is of the cumulative normal distribution and represents the area to the left of the curve for a specified number of standard deviations from the mean. The number of standard deviations equals the sum of the left-hand column and the top row, so, for example, the area for 1.17 standard deviations equals 0.87900.

Using this table, it is then possible to determine the probability of a measurement between any specific limits.

- The probability that a measurement is above 1 standard deviation from the mean is equal to $1 - 0.84134 = 0.15866$.
- The probability that a measurement is more than 1 standard deviation from the mean will be twice this because both positive and negative deviations are possible and the curve is symmetrical and is equal to 0.31732. Put another way, around a third of all measurements will fall outside 1 standard deviation from the mean.
- The probability that a measurement falls between −2 and +1 standard deviations from the mean can be calculated as follows.
  - The probability that a measurement falls between 0 and −2 standard deviations is the same as the probability it falls between 0 and +2 standard deviations and is equal to $0.97725 - 0.5 = 0.47725$.
  - The probability that a measurement falls between 0 and +1 standard deviations is equal to $0.84134 - 0.5 = 0.34134$.
  - Therefore, the total probability is $0.47725 + 0.34134 = 0.81859$.

The normal distribution curve is not only a probability distribution but is also used to describe peak shapes in spectroscopy and chromatography.

Instead of using tables, there are various functions in almost all common packages such as Excel and Matlab to calculate the normal probability distribution. In this section, for brevity, we just describe the use of traditional statistical tables for brevity, for the normal distribution and related distributions. This is expanded in the sections on Excel and Matlab.

A.3.3 χ^2-Distribution

When there is one variable measured, the χ^2 distribution can be very simply related to the normal distribution. χ^2 is simply the square of the distance to the mean. If a series of samples are normally distributed, we can describe the distribution of their squares by χ^2. Therefore, for example, if data are standardised, the proportion of samples less than 4 units of χ^2 from the mean is the same as the proportion of samples within 2 normalised units either side of the mean (i.e. having a value of $z \leq \pm 2$).

The χ^2 distribution becomes important when more than one variable is measured, that is, multivariate data. For the normal distribution, we can look at how far a sample is from the mean in terms of the standard deviation, this may be

There are several uses of χ^2, but in our cases, it is most useful when looking at multivariate distributions, to see whether samples at specified distance from the mean can be regarded as members of the parent distribution or not.

A.3.4 *t*-Distribution

When sample sizes are relatively small, we find that ideal distributions can be distorted. Many early statistical studies used limited sample sizes; thus, it was necessary to develop approaches, not just for estimating means and standard deviations but to deduce more information about the underlying population. Even in more modern studies, some applications such as process control and metabolomics often suffer from modest sample sizes.

The *t*-distribution is similar in nature to the normal distribution but is distorted when the number of degrees of freedom is small. In this context, the number of degrees of freedom (ν) equals the number of samples -1. Once ν reaches about 10, the normal distribution and *t*-distribution are very similar and cannot easily be distinguished. The *t*-distribution can only be used when there is one variable.

Most tables of *t*-distributions present the critical value of the *t*-statistic for different degrees of freedom. Table A.3 is of the two-tailed *t*-distribution, which we employ in this text, and asks whether a parameter differs significantly from another. If there are 10 degrees of freedom and the *t*-statistic equals 2.32, then the probability is quite high, slightly above 95% ($p = 0.05$), that it is, significant (or more accurately a low probability of rejecting the null hypothesis).

From Table A.3, we can see that as the number of degrees of freedom becomes very large, the *t*-distribution converges to the normal distribution.

- The critical value at $p = 0.01$ for ∞ degrees of freedom is 2.576.
- As this is two tailed, it represents all data above 0.995 and below 0.995 standard deviations from the mean.
- From Table A.1, we do see that this is indeed between 2.57 and 2.58 standard deviations from the mean, so the two tables agree.

Note that there are several ways in which the critical values of t can be presented, and some tables are for the one-tailed critical levels, simply double the p values while using the two-tailed critical levels.

The one-tailed test is used to see whether a parameter is significantly larger than another; for example, does the mean of a series of samples significantly exceed that of a series of reference samples? In this book, we are mainly concerned with using the *t*-statistic to determine the significance of coefficients when analysing the results of designed experiments; in such cases, both a negative and positive coefficient are equally significant, so a two-tailed test is most appropriate.

A.3.5 *F*-Distribution

The *t*-distribution cannot be used when there is more than one variable, analogous to the normal distribution. The *F*-distribution can be regarded as the equivalent extension of the *t*-distribution when there is more than one variable but small sample sizes, or analogous to χ^2, but when there are small sample sizes. The *F*-statistic is always positive and represents a squared number.

There are numerous ways of introducing this distribution in the literature, which are widely employed in many diverse areas. The *F*-distribution is often introduced in the context of ANOVA, but can also be used in a modified form (called Hotelling's T^2) to determine the significance of the Mahalanobis distance from the centre of a group in multivariate space.

- The *F*-distribution is characterised by two different types of degrees of freedom.
- It is often written as $F(\nu_1, \nu_2)$.
- In the context of multivariate matrices, if we consider a data set consisting of I samples and J variables, then ν_1 represents the number of variables and ν_2 the number of observations minus the number of variables $(I - J)$. The first degree of freedom represents the number of columns in a data matrix, which is the reverse of the way dimensions are denoted; thus, a multivariate matrix has dimensions $\nu_2 \times (\nu_1 + \nu_2)$.
- The *F*-distribution is often used in a wide number of contexts.
- Note that $F(\nu_1, \nu_2) \neq F(\nu_2, \nu_1)$.

The *F*-test is also commonly used to compare two variances or errors and ask whether either one variance is significantly greater than the other (one-tailed) or whether it differs significantly (two-tailed). In this book, we only use the one-tailed *F*-test, mainly to see whether one error (e.g. lack-of-fit) is significantly greater than a second one (e.g. experimental or analytical).

critical F values for any probability and combination of degrees of freedom, but it is still worth being able to understand the use of tables.

If one error (e.g. the lack-of-fit) is measured using eight degrees of freedom and another error (e.g. replicate) is measured using six degrees of freedom, then if the F-ratio between the mean lack-of-fit and replicate errors is 7.89, is it significant? The critical F-statistic at 1% is 8.1017 and at 5% is 4.1468. Hence, the F-ratio is significant at almost 99% (=100 − 1)% level because 7.89 is almost equal to 8.1017. Thus, we are 99% certain that the lack-of-fit is really significant (or $p < 0.01$).

Some texts also present tables for a two-tailed F-test, but for brevity, we omit this. However, a two-tailed F-statistic at 10% significance is the same as a one-tailed F-statistic at 5% significance, and so on.

The F-distribution can be used in many other contexts, for example, to determine how likely a sample is to belong to a parent distribution if it is a specified Mahalanobis distance from the centre of the distribution, according to the number of samples and variables characterising a training set: however, if the sample size is large, it is also possible to use the χ^2 distribution, and it is unlikely that the difference could be observed experimentally. Note that when dealing with multivariate distances, a scaling factor is usually necessary, resulting in Hotelling's T^2, which has the same shape as F. These measures all do depend on the underlying data being normally distributed.

A.4 Excel for Chemometrics

There are many excellent books on Excel in general, and the package in itself is associated with an extensive help system. It is not the purpose of this text to duplicate these books, which in themselves are regularly updated as new versions of Excel become available, but to primarily indicate the features that the user of advanced data analysis might find useful. The examples in this book are illustrated using Office 365 using Excel 2016 under Windows 10, but most features are applicable to most versions of Excel and are also likely to be upwards compatible. It is recommended to use 32 bit rather than 64 Excel as some macros are not compatible with the 64 bit version. To check whether your version of Excel is 32 bit, open the 'File' tab, then the 'Account' menu item and then the 'About Excel' button. It is assumed that the reader has already some experience in Excel, and the aim of this section is to indicate some features that will be useful to the scientific user of Excel, especially the chemometrician. This section should be regarded primarily as one of tips and hints and the best way forward is by practice. Comprehensive help facilities and websites are also available for the specialist user of Excel. The specific chemometric add-ins available to accompany this text are also described.

A.4.1 Names and Addresses

There are a surprisingly large number of methods for naming cells and portions of spreadsheets in Excel, and it is important to be aware of all the possibilities.

A.4.1.1 Alphanumeric Format

The default naming convention is alphanumeric, each cell's address involving one or two letters referring to the column followed by a number referring to the row. Thus, cell *C5* is the address of the fifth row of the third column. The alphanumeric method is a historic feature of this and most other spreadsheets but is at odds with the normal scientific convention of quoting rows before columns. After the letters A–Z are exhausted (the first 26 columns), two-letter names are used starting at AA, AB, AC up to AZ and then BA, BB, then after ZZ, AAA, AAB and so on.

A.4.1.2 Maximum Size

The size of a worksheet is limited, to a maximum of 16 384 (=2^{14}) columns and 1 048 576 (=2^{20}) rows, which means the highest address is *XFD*1048576. There is no difference in size between 32 bit and 64 bit Excel in modern implementations providing Windows is 64 bit. 64 bit Excel is faster when data sets are very large, but still has disadvantages, in that macros and add-ins are not upward compatible.

A.4.1.3 Numeric Format

Some people prefer using a numeric format for addressing cells. The columns are numbered rather than labelled with letters. To change from the default, select the 'File' tab, choose 'Options' menu item, then 'Formulas' menu item and select *R1C1* checkbox (Figure A.1). Deselecting this returns to the alphanumeric system. When using the RC notation, the rows are cited first, as is normal for matrices, and the columns are cited second, which is the opposite to the default convention. Cell *B5* is now called cell *R5C2*. Below we employ the alphanumeric notation unless specifically indicated.

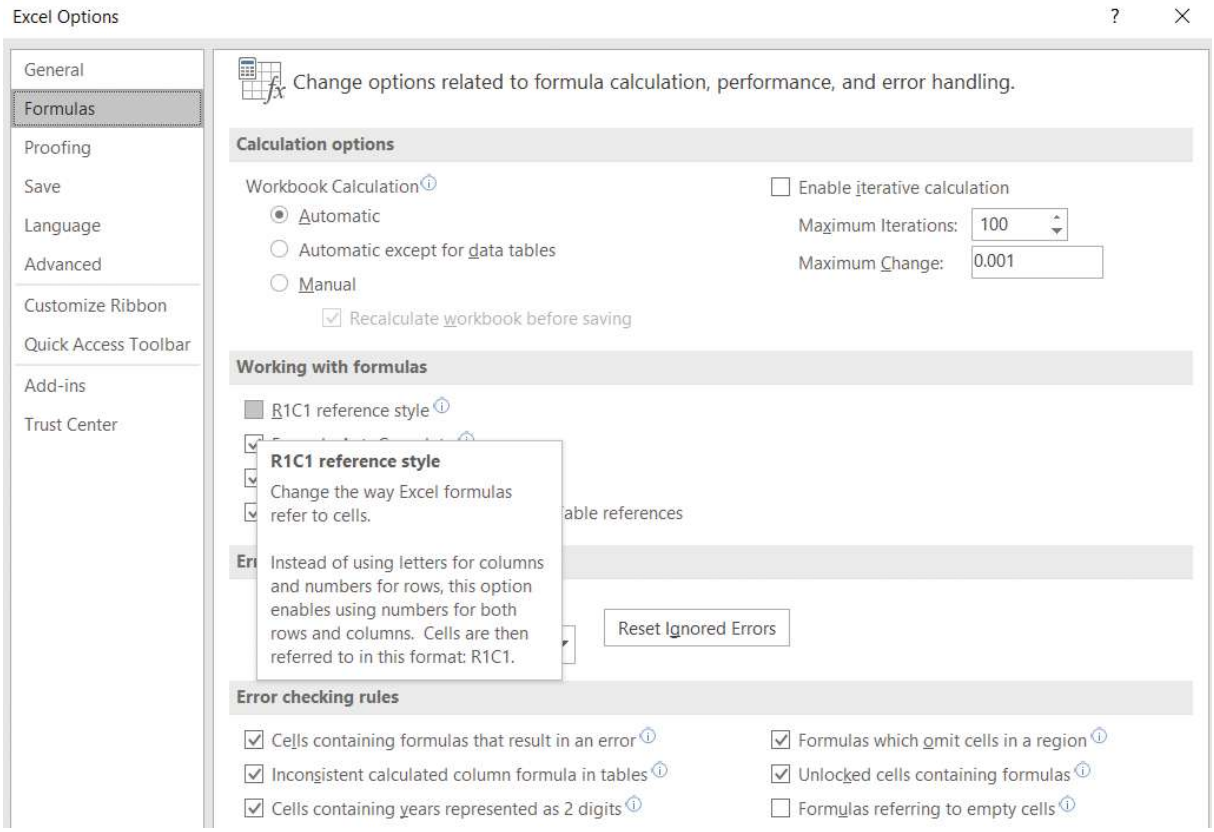


Figure A.1 Changing to numeric cell addresses.

A.4.1.4 Worksheets

Each spreadsheet file can contain several worksheets, each of which either contain data or graphs and which can also be named. The default is *Sheet1*, *Sheet2* and so on, but it is possible to call the sheets by almost any name, such as *Data*, *Results* and *Spectra*, and it is also possible to contain spaces, for example, *Sample 1A*. A cell specific to one sheet has the address of the sheet included, separated by a !; hence, the cell *Data!E3* is the address of the third row and fifth column on a worksheet *Data*. This address can be used in any worksheet, thus allows results of calculations to be placed in different worksheets. If the name contains a space, it is cited in quotes, for example, *'Spectrum 1'!B12*.

New sheets can be inserted using the Home tab, and cells ribbon, select 'Insert Sheet', so any number of sheets can be inserted this way and named as appropriate.

A.4.1.5 Spreadsheets

In addition, it is possible to reference data in another spreadsheet file, for example *[First.xlsx]Result!A3* refers to the address of cell *A3* in worksheet *Result* and file *First.xlsx*. This might be potentially useful, for example, if the file *First* consists of a series of spectra or a chromatogram, whereas the current file consists of the results of processing the data, such as graphs or statistics. This flexibility comes with a price as all files have to be available simultaneously and is somewhat elaborate especially in cases where files are regularly reorganised and backed up.

Note that there are several file extensions for Excel, the commonest being *.xls* (old versions of Excel), *.xlsx* (default) and *.xlsm* (contains macros). If you get into a muddle, always check the file extension, which unless specified by you will be a default or previously used extension. While referring to other spreadsheets, you must include the full extension to the filename.

A.4.1.6 Invariant Addresses

When copying an equation or a cell address within a spreadsheet, it is worth remembering another convention. The $ sign means that the row or column is invariant. For example, if the equation '=A1' is placed in cell *C2*, and then this is

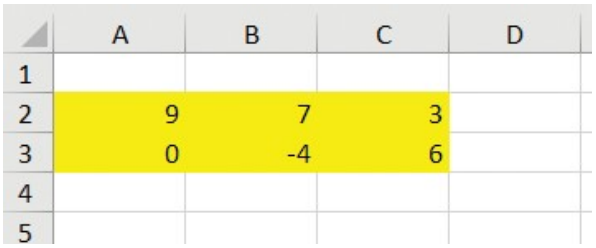
copied to cell *D4*, the relative references to the rows and columns are also moved; hence, cell *D4* will actually contain the contents of the original cell *B3*. This is often quite useful because it allows operations on entire rows and columns, but sometimes we need to avoid this. Placing the equation '=$A1' in cell *C2* has the effect of fixing the column, so that the contents of cell *D4* will now be equal to those of cell *A3*. Placing '=A$1' in cell *C2* makes the contents of cell *D4* equal to *B1*, and placing '=$A$1' in cell *C2* makes cell *D4* equal to *A1*. The naming conventions can be combined, for example, *Data!B$3*. Experienced users of Excel often combine a variety of different tricks and it is best to learn these ways by practice rather than reading lots of books.

A.4.1.7 Ranges

A range is a set of cells, often organised as a matrix. Hence, the range *A2:C3* consists of six numbers organised into two rows and three columns, as illustrated in Figure A.2. It is possible to calculate the function of a range, for example *=AVERAGE(A2:C3)*, these will be described in more detail in Section A.4.2.4. If this function is placed in cell *D4*, then if it is copied to another cell, the range will alter correspondingly; for example, moving from cell *D4* to cell *E6* would change the range, automatically, to *B4:D5*. There are various ways to overcome this, the simplest being to use the $ convention as described above. Note that ! and [] can also be used in the address of a range, so that *=AVERAGE([first.xls]Data!$B$3:$C$10)* is an entirely legitimate statement.

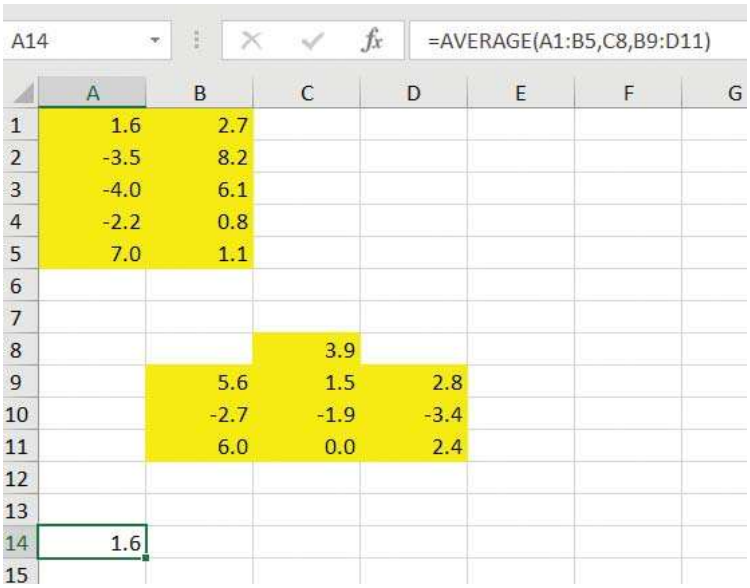
It is not necessary for a range to be a single contiguous matrix. The statement *=AVERAGE(A1:B5,C8,B9:D11)* will involve 10 + 1 + 9 = 20 different cells as indicated in Figure A.3.

The references to cells or ranges can be dragged and remain invariant. If the middle of a cell is 'hit', the cursor turns into an arrow. Dragging this drags the cell, but the original reference remains unchanged, see Figure A.4, where the



	A	B	C	D
1				
2	9	7	3	
3	0	-4	6	
4				
5				

Figure A.2 The range A2:C3.



A14 | =AVERAGE(A1:B5,C8,B9:D11)

	A	B	C	D	E	F	G
1	1.6	2.7					
2	-3.5	8.2					
3	-4.0	6.1					
4	-2.2	0.8					
5	7.0	1.1					
6							
7							
8			3.9				
9		5.6	1.5	2.8			
10		-2.7	-1.9	-3.4			
11		6.0	0.0	2.4			
12							
13							
14	1.6						
15							

Figure A.3 The operation =AVERAGE(A1:B5,C8,B9:D11).

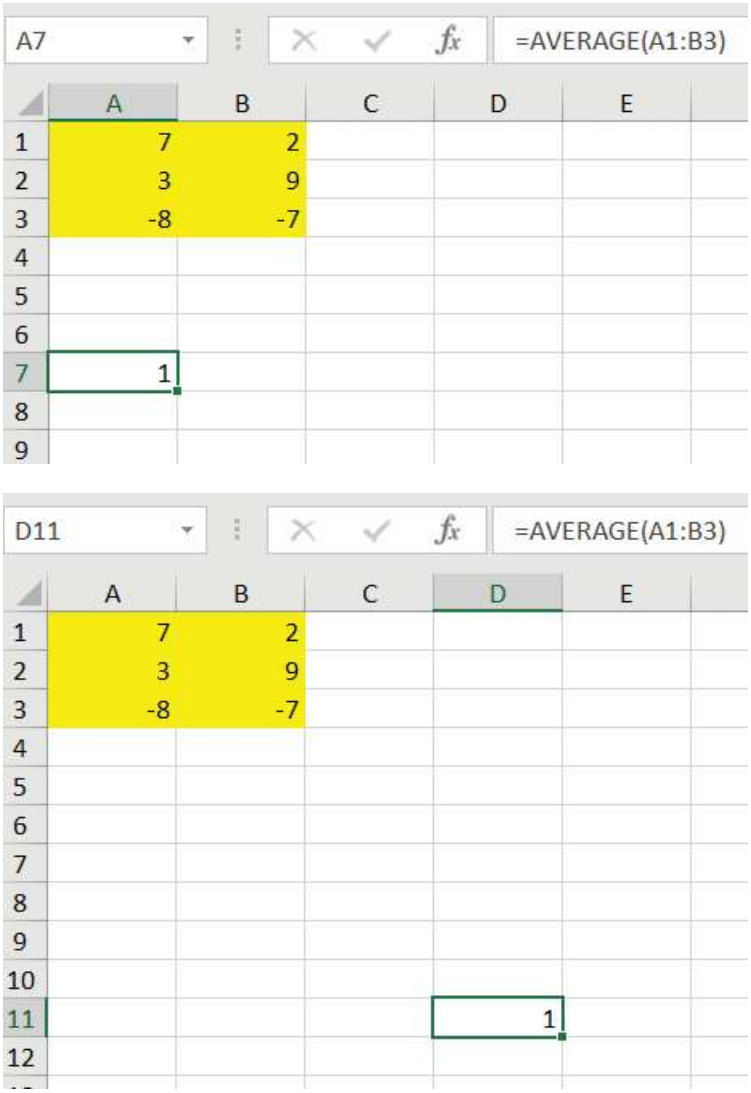


Figure A.4 Dragging a cell so that the reference is invariant.

equation *=AVERAGE(A1:B3)* had been placed originally in cell *A7*. If a cell is copied, however, the relative references change unless the $ sign has been used as above.

A.4.1.8 Naming Matrices

It is particularly useful to be able to name matrices or vectors. This is quite straightforward. Using the 'Formulas' ribbon and choosing 'Name Manager', it is possible to select a portion of a worksheet and give it a name. Figure A.5 shows how to call the range A1–B3 'X', creating a 3×2 matrix. It is then possible to perform operations on these matrices, so that *=SUMSQ(X)* gives the sum of squares of the elements of X and is an alternative to *=SUMSQ(A1:B3)*. It is possible to perform matrix operations; for example, if both X and Y are 4×2 matrices, select a third 4×2 region and place the command $=X+Y$ in that region (as discussed in Section A.4.2.2, it is necessary to end all matrix operations by simultaneously pressing the <SHIFT> <CTRL> <ENTER> keys). Quite elaborate commands can then be nested; for example, $=3*(X+Y)-Z$ is entirely acceptable where all the three arrays are named matrices, notice that the '3*' is a scalar multiplication: more about this will be discussed below.

The name of a matrix is common to an entire spreadsheet, rather than any individual worksheet. This means that it is possible to store a matrix called 'Data' on one worksheet and then perform operations on it in a separate worksheet.

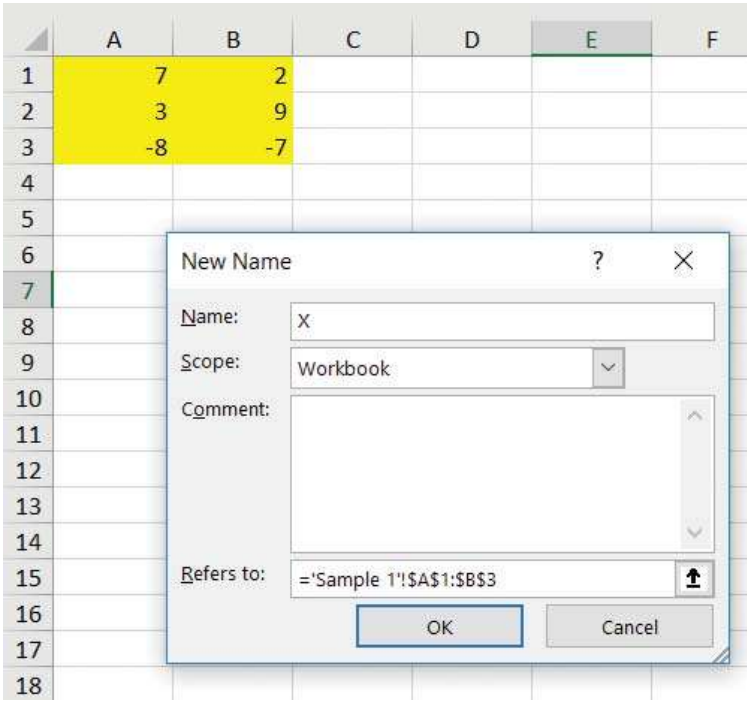


Figure A.5 Naming a range.

A.4.2 Equations and Functions

A.4.2.1 Scalar Operations

The operations are straightforward and can be performed on cells, numbers and functions. The operations +, −, * and / indicate addition, subtraction, multiplication and division as in most environments; powers are indicated by ^. Brackets can be used and there is no practicable limit to the size of the expression or the amount of nesting. A destination cell must be chosen. Hence, the operation $=3*(2-5)\text{^}2/(8-1)+6$ gives a value of 9.857. All the usual rules of precedence are obeyed. Operations can mix cells, functions and numbers, for example $=4*A1+SUM(E1:F3)-SUMSQ(Y)$ involves adding

- four times the value of cell *A1* to
- the sum of the values of the six cells in the range *E1–F3* and
- subtracting the sum of the squares of the matrix $\boldsymbol{Y}$.

Equations can be copied around the spreadsheet, the $, ! and [] conventions being used if appropriate.

A.4.2.2 Matrix Operations

There are a number of matrix operations, of especial use in chemometrics. These operations must be terminated by simultaneously pressing the <SHIFT>, <CTRL> and <ENTER> keys. It is first necessary to select a portion of the spreadsheet where the destination matrix will be displayed. The most useful functions are as follows.

- *MMULT* multiplies two matrices together. The inner dimensions must be identical. If not, there is an error message. The destination (third) matrix must be selected; if its dimensions are wrong, a result is still given, but some numbers may be missing or duplicated. The syntax is = *MMULT(A,B)* and Figure A.6 illustrates the result of multiplying a 4×3 matrix with a 3×2 matrix to give a 4×2 matrix.
- *TRANSPOSE* gives the transpose of a matrix. Select the destination of the correct shape and use the syntax =*TRANSPOSE(A)* as illustrated in Figure A.7.
- *MINVERSE* is an operation that can only be performed on square matrices and gives the inverse. An error message is presented if the matrix has no inverse or is not square. The syntax is = *MINVERSE(A)* as illustrated in Figure A.8.

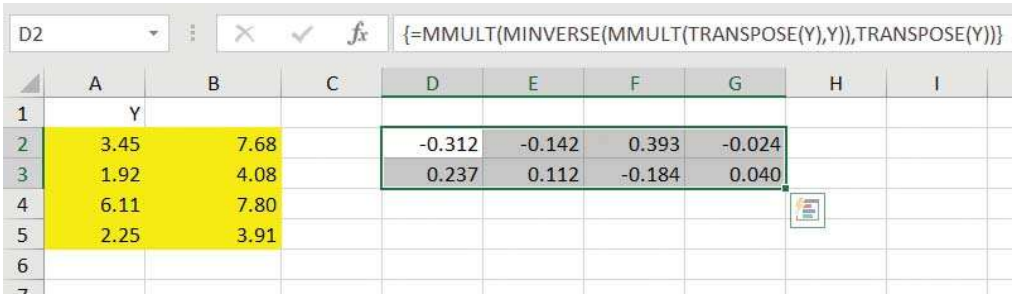


Figure A.9 Pseudo-inverse of a matrix.

It is possible to add and subtract matrices using + and −, but remember to ensure that the two (or more) matrices have the same dimensions as has the destination. It is also possible to mix matrix and scalar operations, so that the syntax *=2*MMULT(X,Y)* is acceptable. Remember always to complete the statement by <SHIFT> <CTRL> <ENTER> if performing arithmetic on matrices rather than scalars. Furthermore, it is possible to add (or subtract) matrices consisting of a constant number, for example, $=Y+2$ would add 2 to each element of Y. Other conventions for mixing matrix and scalar variables and operations can be deduced by practice, although in most cases, the result is what we would logically expect, and for brevity, we will not provide a comprehensive review in this text.

There are a number of other matrix functions in Excel, for example, to calculate determinant and trace of a matrix, to use these select the 'Formulas' tab and 'Math and Trig' menu or use the Help system.

A.4.2.3 Arithmetic Functions of Scalars

There are numerous arithmetic functions that can be performed on single numbers. Useful examples are *SQRT* (square root), *LOG* (log to the base 10), *LN* (natural log), *EXP* (exponential) and *ABS* (absolute value), for example *=SQRT(A1+2*B1)*. A few functions have no number to operate on, such as *ROW()* which is the row number of a cell, *COLUMN()* the column number of a cell and *PI()* the number π. Trigonometric functions operate on angles in radians; thus, be sure to convert if your original numbers are in degrees or cycles, for example *=COS(PI())* gives a value of −1.

A.4.2.4 Arithmetic Functions of Ranges and Matrices

It is often useful to calculate the function of a range; for example, *=SUM(A1:C9)* is the sum of the 27 numbers within the range. It is possible to use matrix notation so that *=AVERAGE(X)* is the average of all the numbers within the matrix X. Notice that as the answer is a scalar, these functions are not terminated by <SHIFT> <CTRL> <ENTER>. Useful functions include *SUM*, *AVERAGE*, *SUMSQ* (sum of squares) and *MEDIAN*.

Some functions require more than one range. The *CORREL* function is useful for the chemometrician and is used to compute the correlation coefficient between two arrays, syntax *=CORREL(A,B)*, and is illustrated in Figure A.10. Another couple of useful functions involve linear regression. The functions *=INTERCEPT(Y,X)* and *=SLOPE(Y,X)* provide the parameters b_0 and b_1 in the equation $y = b_0 + b_1 x$, as illustrated in Figure A.11.

Standard deviations, variances and covariances are useful common functions. It is important to recognise that there are both population and sample functions, so that *STDEV.S* is the sample standard deviation and *STDEV.P* the

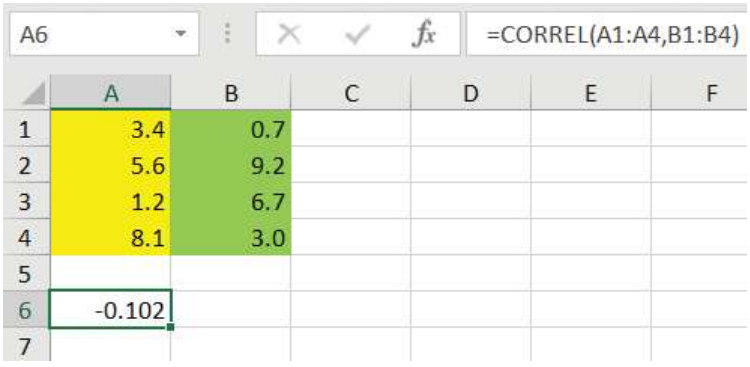


Figure A.10 Correlation between two ranges.

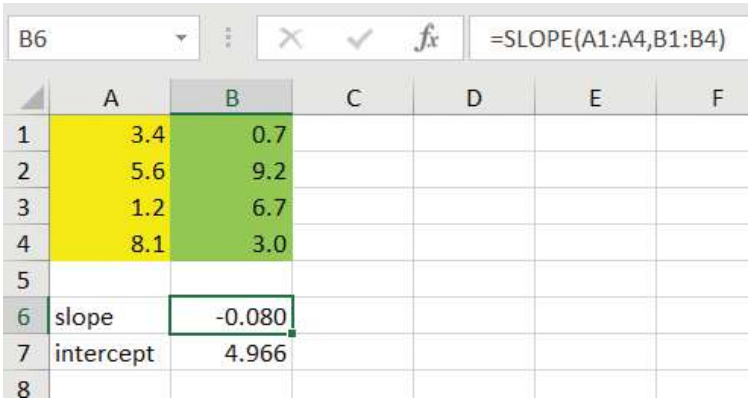


Figure A.11 Finding the slope and intercept when fitting a linear model to two ranges.

equivalent population standard deviation. Note that for standardising matrices, it is a normal convention to use the population standard deviation. Similar comments apply to *VAR.S* and *VAR.P*. For the covariance, use *COVARIANCE.S* and *COVARIANCE.P*. There are legacy functions from previous versions of Excel, which are not so logical, be careful if choosing these functions.

A.4.2.5 Statistical Functions

Surprisingly, a large number of common statistical functions are available in Excel. Conventionally, many such functions are presented in tabular format as in this book, for completeness, but most information can be easily obtained from Excel.

The inverse normal distribution is quite useful and allows a determination of the number of standard deviations from the mean to give a defined probability; for example, *=NORM.INV(0.9,0,1)* is the value within which 90% (0.9) of the readings will fall if the mean is 0 and standard deviation is 1, and equals 1.282, which can be verified using Table A.1. The function *NORM.DIST* returns the probability of lying within a particular value; for example, *=NORM.DIST(1.5,0,1,TRUE)* is the probability of a value that is less than 1.5, for a mean of 0 and a standard deviation of 1, using the cumulative normal distribution (=*TRUE*), and equals 0.99319 (see Table A.1). Similar functions such as *T.DIST*, *T.INV*, *CHISQ.DIST*, *CHISQ.INV*, *F.DIST* and *F.INV* can be employed if required, eliminating the need for tables, although these do not allow a standard deviation or mean to be specified, but require the relevant degrees of freedom to be specified. It is important to check whether the functions are one tailed or two tailed if appropriate, or right side/left side of the distribution as required and there are several alternatives: the reader is advised to read the description of the functions in Excel when first using them. There are also a likewise number of legacy functions to be compatible with early versions of Excel.

A.4.2.6 Logical Functions

There are several useful logical functions in Excel. *IF* is a common one. Figure A.12 is of the function *=IF(A1<B1,A1,B1)* and places the lower of the values of columns A and B in column D. Notice that this has been copied down the column

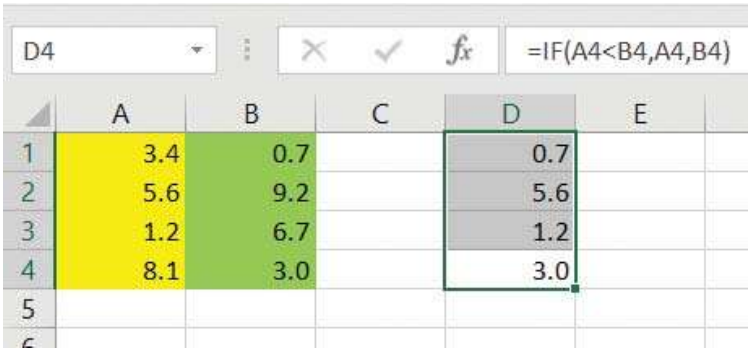


Figure A.12 Use of IF in Excel.

and also that there are no $ signs in the arguments in this case. *COUNTIF* can be used to determine how many times an expression is valid within a region of a worksheet. This is quite useful, for example, to determine how many values of a matrix are above a threshold or equal zero.

A.4.2.7 Nesting and Combining Functions and Equations

It is possible to nest and combine functions and equations. The expression *=$C6+IF(A$7>1,10,IF(B$3*$C$2>5, 15,0))^2-2*SUMSQ(X+Y)* is entirely legitimate, although it is important to ensure that each part of the expression results in compatible type of information (in this case, the result of using the *IF* function is a numerical value that is squared). Note that spreadsheets are not restricted to containing numerical information, they may, for example, also contain names (characters) or logical variables or dates, and some functions will operate on non-numerical information. In this section, we have concentrated primarily on numerical functions, as these are the most useful for the chemometrician, but it is important to recognise that nonsensical results would be obtained, for example, if trying to add a character to a numerical expression to a date.

A.4.3 Add-Ins

A very important capability of Excel consists of add-ins. In this section, we will describe only those add-ins that are part of the standard Excel package. It is possible to write one's own add-ins or download a number of useful add-ins from the web. This book is associated with an add-ins specifically for chemometrics, as will be described in Section A.4.5.2.

If properly installed, there should be a 'Data Analysis' item in the 'Data' ribbon. If this does not appear, you should select the 'File' menu, then select 'Options' and the 'Add-ins' and the find 'Manage Excel Add-ins', as illustrated in Figure A.13(a). Select 'Go' and then you will be presented with a list of add-ins, which will depend on your installation, and select the 'Analysis Toolpak' typically illustrated in Figure A.13(b). One occasional problem is that some institutes use Excel over a network. The problem with this is that it is not always possible to install these facilities on an individual computer, but this must be performed by the Network administrator, dependent on your configuration.

Once the menu item is selected, the dialog box of Figure A.14 should appear. There are several useful facilities, but probably, the most important for the purpose of chemometrics is the 'Regression' feature. The default notation in Excel differs from that in this book. A multiple linear model is formed between a single response y and any number of x variables. Figure A.15 illustrates the result of performing regression on one x variable to give the best fit model $y \approx b_0 + b_1 x_1 + b_2 x_2$. There are quite a number of statistics produced. Notice in the dialog box one selects 'constant is zero' if one does not want to have a b_0 term, this is equivalent to forcing the intercept to be equal to 0. The answer, in the case illustrated, is $y \approx -0.15762 + 0.29993\ x_1$, see cells *B30–B31*. Notice that this answer could also have been performed using matrix manipulations with the pseudo-inverse, after first adding a column of 1s to the $\boldsymbol{X}$ matrix, as described in Section A.1.2.5 and elsewhere. There are quite a few options in the regression function, which the interested reader can gain experience of, if required.

A second facility that is sometimes useful is the random number generator function. There are several possible distributions, but the most usual is the normal distribution. It is necessary to specify a mean and standard deviation. If one wants to be able to return to the distribution later, also specify a seed, which must be an integer number. Figure A.16 illustrates the generation of 10 random numbers coming from a distribution of mean 6 and standard deviation 2 placed in cells *A1–A10* (note that the standard deviation is of the parent population and will not be exactly the same for a sample). This facility is very helpful in simulations and can be employed to study the effect of noise on a data set.

The 'Correlation' facility that allows one to determine the correlation coefficients between either rows or columns of a matrix is also useful in chemometrics, for example, as the first step in cluster analysis. Note that for two columns, it is better to use the *CORREL* function, but when there are several columns (or variables), the Data Analysis Add-in is easier.

A.4.4 Charts

Most graphs in this text have been produced in Excel, and all graphs from the problems at the end of each chapter can be produced either in Excel or in Matlab. The graphics facilities are quite good except for 3D representations. This section will briefly outline some of the main features of the chart tool useful for applications in this text.

Graphs can be produced either by selecting the 'Insert' tab and the 'Charts' group. Most graphs in this book are produced using an *xy* or Scatter plot, allowing the value of one parameter (e.g. the score of PC2) to be plotted against

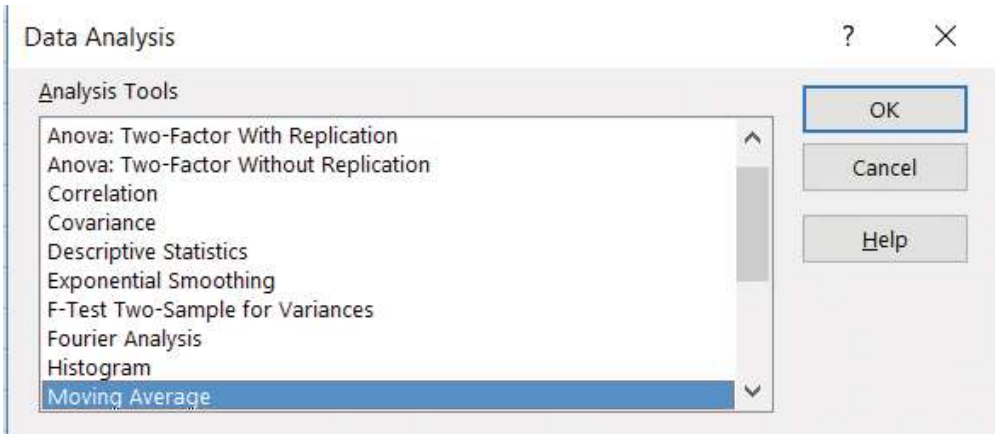


Figure A.14 Data Analysis Add-in dialog box.

another (e.g. the score of PC1). Various enhancements to the appearance of graphs can be learnt by experience and we will not list all these in detail for brevity.

It is often desirable to use different symbols for groups of parameters or classes of compounds. This can be done by superimposing several graphs, each being represented by a separate series. This is illustrated in Figure A.17 in which cells *A1–B10* represent Series 1 and *A11–B20* represent Series 2, each set of measurements having a different symbol. To achieve this, once a chart has been developed for Series 1, go to the 'Chart Tools' tab and then select the 'Design' tab. You will find a 'Select Data' item, and this allows you to add as many additional series as you like. Each will be default and have different symbols, although, of courses, you can change the symbols for each series as you wish.

The default graphics options are not necessarily the most appropriate and are designed primarily for display on a screen. Typically, we often remove gridlines and remove the box around the graph. In addition, often it is a good idea to label the axes and add a title and increase the font size. These facilities are available and a case of personal preference. The default appearance of Excel Charts has improved considerably over the years.

It is also possible to place the chart on a sheet of its own. In the 'Design Tab', choose 'Move Chart' and select 'Location'. A typical finalised chart is illustrated in Figure A.18.

There are numerous resources on the web to learn more about how to create charts in Excel if desired. One difficulty involves attaching a label to each point in a chart; this may be the name of an object or a variable. With this text, we produce a downloadable macro that can be edited to permit this facility, as described in Section A.4.5 There are, of course, also numerous facilities available on the web as alternatives.

A.4.5 Downloadable Macros

To facilitate the use of charts and also common multivariate methods, we provide some downloadable macros in the companion website. Macros are programs written in a language called VBA that can be run in Excel to enhance the normal facilities. For more details, read the Excel literature or search the web.

A.4.5.1 Labelling Macro

A macro that allows points to be added to be labelled in a scatterplot is available.

The simplest approach is to download the file 'label', which is in the old Excel format to be compatible with previous versions and open it in the same directory as your current file that contains your data. In the file containing your data, you should have a 'Developer' tab, which can be opened, to reveal a 'Macros' item, which if clicked open will reveal some macros that have been pre-written. If there is no 'Developer' tab, go to the 'File' tab, then 'Options' menu item and then select 'Customise Ribbon' and ensure you can add the 'Developer' tab. You should see the `AddChartLabels` macro that allows labels to be added to points in a graph.

First, produce a scatterplot in Excel using the Chart Wizard. Make sure that there are two columns to the left of the data in Excel. The first should contain the labels you want for each point. The second (on the far left) should contain an 'x' for each point you want to label. This allows you to select points on the graph. If you want to label all the points, then put at 'x' all the way down the column, see Figure A.19. Select the graph. Then, simply run the macro, you will be

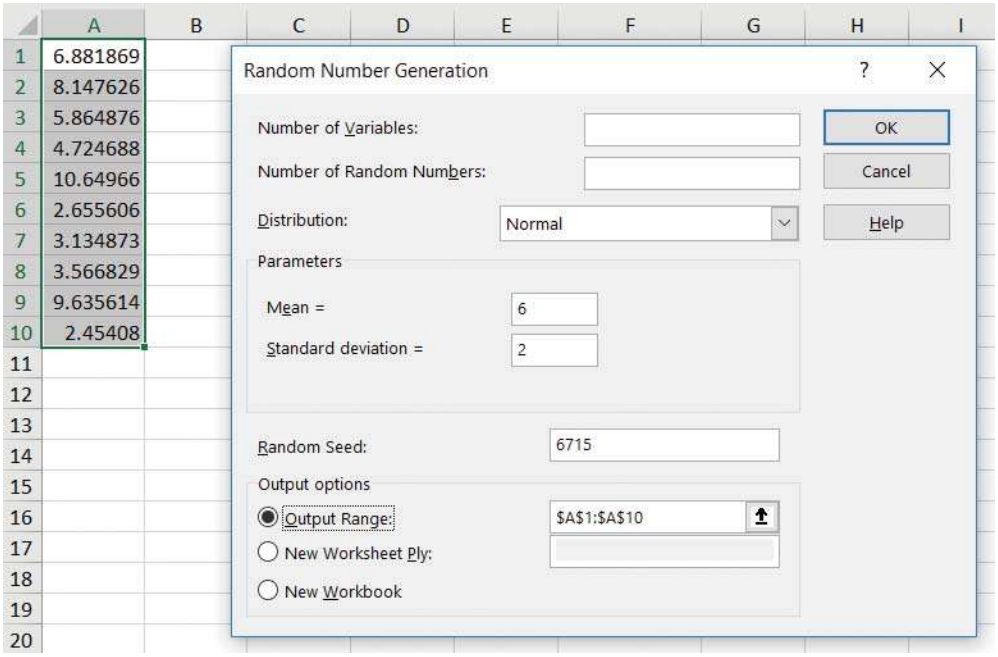


Figure A.16 Generating random numbers in Excel.

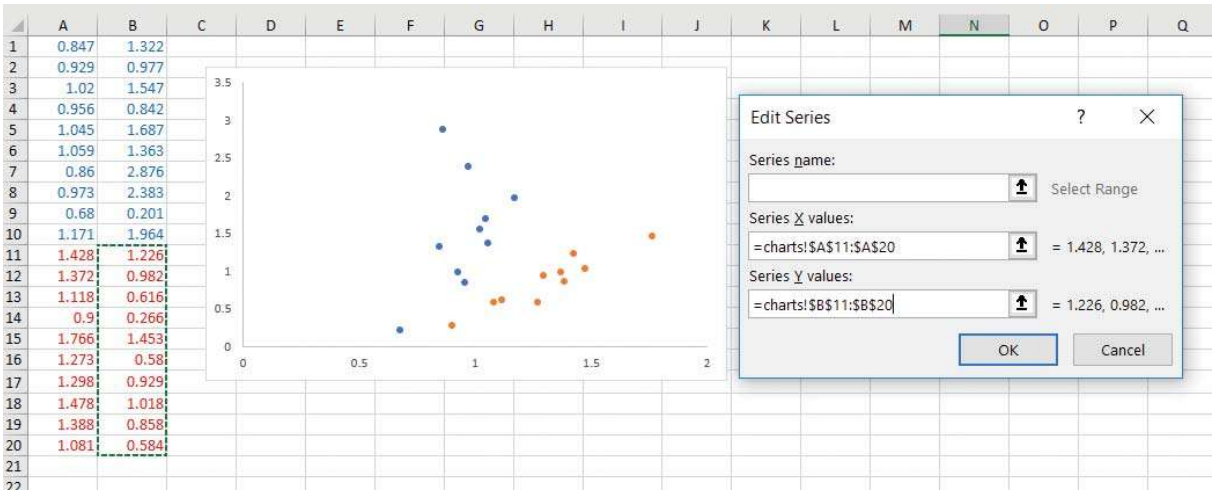


Figure A.17 Adding an extra series in Excel.

asked the font size and then each point should be labelled as in the figure. If some of the labels overlap after the macro has been run, for example, if there are close points in a graph, you can manually select each label and move it around the graph, or even delete selective labels or change the colour of individual labels, for example, to emphasize specific data points.

A.4.5.2 Multivariate Analysis Add-In

Accompanying the text is also an add-in to perform several methods for multivariate analysis. The reader is urged first to understand the methods by using matrix commands in Excel or Matlab scripts, and several examples in this book guide the reader to understanding these methods from scratch. However, after doing this once, it is probably unnecessary to repeat the full calculations from scratch and convenient to have available add-ins in Excel. Although the performance has been tested on computers of a variety of configurations, the software was originally developed for Office 2000 and Windows 98, using 64 MB memory, which almost all present-day computers exceed. There may

must be selected. This facility also performs regression using the pseudo-inverse and is mainly provided for completion. Note that it is not necessary to restrict the data to spectra or concentrations. MLR can also be performed using standard matrix operations in Excel as described in Section A.4.2.2.

This add-in provides a basic functionality for many of the multivariate methods described in Chapters 4–7 and can be used when solving the problems at the end of each chapter in Excel.

A.5 Matlab for Chemometrics

Many chemometricians use Matlab. In order to appreciate the popularity of this approach, it is important to understand the vintage of chemometrics. The first applications of quantum chemistry, another type of computational chemistry, were developed in the 1960s and 1970s where Fortran was the main numerical programming environment. Hence, large libraries of routines were established over this period, and to this day, most quantum chemists still program in Fortran. Were the disciplines of quantum chemistry to start over again, probably Fortran would not be the main programming environment of choice, but tens of thousands (or more) man-years would need to be invested to rewrite the entire historic databases of programs. If we were developing an operating system that would be used by tens or hundreds of millions of people, that investment might be worthwhile, but the scientific market is much smaller; hence, once the environment is established, new researchers tend to stick to it as they can then exchange code and access libraries.

Although some early chemometrics code was developed in Fortran (the Arthur package of Kowalski) and Basic (Wold's early version of SIMCA) and commercial packages are mainly written in C, most public domain chemometrics code first became available in the 1980s where Matlab was an upcoming new environment. An advantage of Matlab is that it is very much oriented towards matrix operations and most chemometrics algorithms are best expressed this way. It can be awkward to write matrix-based programs in C, Basic or Fortran unless one has access to or develops specialised libraries. Matlab was originally a technical programming environment mainly for engineers and physical scientists, but over the years, the user base has expanded strongly and Matlab has kept pace with new technology including extensive graphics, interface to Excel, numerous toolboxes for specialist use and the ability to compile software. In this section, we will primarily concentrate on the basics required for chemometrics and also to solve the problems in this book; for the more experienced user, there are numerous other outstanding texts on Matlab, including the extensive documentation produced by the developer of the software, the MathWorks, which maintains an excellent website. In this book, you will be introduced to a number of main features to help you solve the problems, but as you gain experience, you will undoubtedly develop your own personal favourite approaches. Matlab can be used at many levels, and it is now possible to develop sophisticated packages with good graphics in this environment.

There are many versions of Matlab and of Windows, and for the more elaborate interface between the two packages, it is necessary to refer to technical manuals. We will illustrate this section with Matlab R2016a running under Windows 10, although some readers may have access to more up-to-date editions. Most facilities are forward compatible, although there are a few small deletions of commands, so be careful if you try to run elaborate 20-year-old code. There is quite a good online help facility in Matlab, you can type `help` followed by the command, or else click on the '?' icon on the top right of the screen to search documentation. However, it is useful to first have a grasp of the basics, which will be described below. We will primarily describe the main commands necessary for the sort of chemometric calculations described in this book and introduce methods in their simplest form: more experienced users will be able to expand on these basics, and this Appendix is not intended as a comprehensive reference, the user to get more familiar and to provide some tricks of the trade specifically useful for the chemometrics expert as an aid to producing graphs and performing calculations relevant to this book. It should be regarded as a dip in the water, once aided, next steps should be independent.

In addition to having access to core Matlab, it is useful to have access to a number of toolboxes. For this book, we will assume that the reader has access to the 'Statistics and Machine Learning' Toolbox as well. There are numerous toolboxes, but this one is the most important for the chemometrics expert.

A.5.1 Getting Started

To start Matlab, it is easiest to simply click the icon that should be available if properly installed, the first time you use Matlab, and a blank screen as in Figure A.27 will appear. It is probably wise to first change the directory you would like to work in: this will be the directory files are read from and saved to and can be done by navigating through the file icon on the top left corner. You can always change this directory at any stage and also set up a default start up

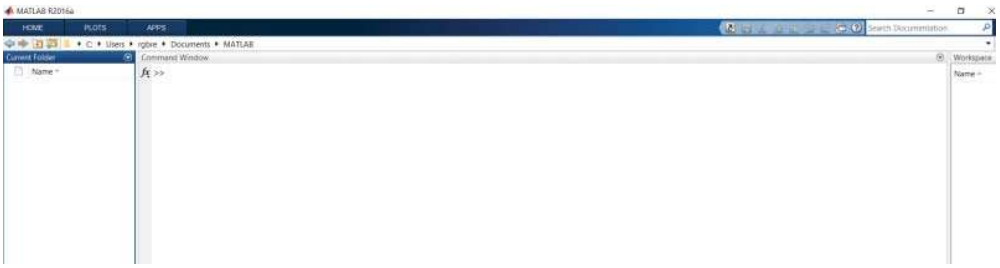


Figure A.27 Default Matlab window.

directory should you so wish. If in a script (or program) you want to change the directory name, use the `cd` command, but remember that a script is often designed to be used on different computers; hence, in many cases, this is unwise.

To enter commands, find the main Command window, where you can type code in. The easiest way to start is by typing commands in after the '≫' prompt.Each Matlab command is typed on a separate line, terminated by the <ENTER> key. If the <ENTER> key is preceded by a semicolon (;), there is no output from Matlab (unless you have made an error) and on the next line you type the next command and so on. Otherwise, you are given some output, for example the result of multiplying matrices together, which can be useful, but if the information contains several lines of numbers that fill up a screen and which may not be very interesting, it is best to suppress this. Hence, if writing a long program (or script in Matlab terminology), if you wish to supress output except when you choose, terminate all statements with a semicolon.

Matlab is case sensitive; hence, the variable x is different to X. Commands are all lower case.

A.5.2 File Types

There are several types of files that one may wish to create and use, but there are three main kinds that are useful for the beginner.

A.5.2.1 Mat Files

These files store the 'workspace' or variables created during a session. All matrices, vectors and scalars with unique names are saved. Many chemometricians exchange data in this format. The command `save` places all this information into a file called `matlab.mat` in the current working directory. If you wish to save into a named file, type a filename, so the command `save mydata` saves the workspace into a file called `mydata.mat` in the current directory. If you want a space in the filename, enclose in single quotes for example `'Tuesday file'`.

In order to access these data in Matlab from an existing file, for example, you may have done some calculations a few days ago, or stored some spectra, simply use the `load` command, remembering what directory you are in, for example, type `load mydata`. If only one or two arrays are required, typing the array names after the filename will just load these, for example, `load mydata X`, just loads array $\boldsymbol{X}$.

Alternatively, on the left (using the default window) is a list of files in the current directory and below that, if a mat file is selected, a list of arrays. Clicking either the filename (to input all arrays) or an individual array name will read these data into Matlab. This is illustrated in Figure A.28.

A.5.2.2 m Files

Quite often, it is useful to create programs that can be run again. This is done via m files. The same rules about directories apply as discussed above.

These files are simple text files and may be created in a variety of ways. A simple way of creating an m file is to click on the Home menu item on the main Matlab window, then 'New' and 'Script'. You can then type commands into an Editor window, and when you are happy you can save it, and if you wish, close this window again.

There are various ways in which you can run the script. For the simplest implementation, make sure everything is in the same directory. Then in the Matlab Command window, you can just type the name of the m file, for example, `Simplecode`, as shown in Figure A.29.

Some m files are functions: to create such an m file, after 'New', select 'Function'. These differ slightly from straight scripts, in that they usually involve the input of one or more arrays and the output of one or more arrays, although functions without a specified input or output (the function could, example.g. create a graph) are legitimate. Consider a simple function `codeforsum`. This takes a matrix and returns the sum of all its elements. The syntax can be written

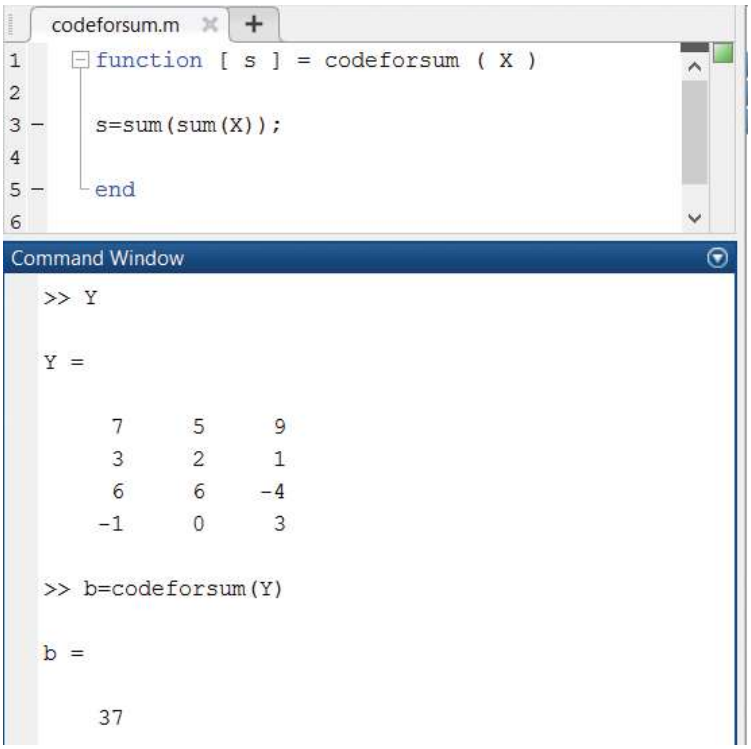


Figure A.30 Running an m file function in Matlab.

The experienced Matlab user can establish libraries of m files and use these in different applications. Different programmers have different styles. Equally some programmers just like running all their code in the Command window. For most of the problems in this text, only a few statements are required, so the code can be either developed as one single file, even copied and pasted into the Command window, or as interactive Matlab commands. Obviously, if you decide to use Matlab for your personal research, or even to swap code with colleagues, you are likely to want to develop more elaborate habits.

A.5.2.3 Diary Files

These files keep a record of a session. The simplest approach is not to use diary files but just to copy and paste the text of a Matlab session, but diary files can be useful because one can selectively save just certain commands. In order to start a diary file, type `diary` (a default file called `diary` will be created in the current directory) or `diary filename` where `filename` is the name of the file. This automatically opens a file into which all subsequent commands used in a session, together with their results, are stored. To stop recording, simply type `diary off` and to start again (in the same file) type `diary on`.

The file can be viewed as a text file, in the Text Editor. Note that you must close the diary session before the information is saved.

A.5.3 Matrices

The key to Matlab is matrices. Understanding how Matlab copes with matrices is essential for the user of this environment.

A.5.3.1 Scalars, Vectors and Matrices

It is possible to handle scalars, vectors and matrices in Matlab. The package automatically determines the nature of a variable when first introduced. A scalar is simply a number so

```
X = 2
```

A.5.3.2 Basic Matrix Operations

The basic matrix operations +, − and * correspond to the normal matrix addition, subtraction and multiplication (using the dot product); for scalars, these are also defined in the usual way with the addition of the / symbol for division. For the first two operations, the two matrices should have the same dimensions, and for multiplication, the number of columns of the first matrix should equal the number of rows of the second matrix. It is possible to place the results in a target or else simply display them on the screen as a default variable called `ans`. Figure A.32 illustrates setting up three matrices, a 3×2 matrix $\boldsymbol{X}$, a 2×3 matrix $\boldsymbol{Y}$ and a 3×3 matrix $\boldsymbol{Z}$ and calculating $\boldsymbol{X} * \boldsymbol{Y} + \boldsymbol{Z}$.

There are quite a number of elaborations based on these basic operations, but the first time user is recommended to keep things simple. However, it is worth noting that it is possible to add scalars to matrices. An example involves adding the number 2 to each element of W, as defined above, either type `W + 2` or first define a scalar (e.g. `P = 2`) and then add this using the command `W + P`. Similarly, one can multiply, subtract or divide all elements of a matrix by a scalar. Notice that it is not possible to add a vector to a matrix even if the vector has one dimension identical to that of the matrix.

A.5.3.3 Matrix Functions

A significant advantage of Matlab is that there are several further very useful matrix operations. Most are in the form of functions; the arguments are enclosed in brackets. Three arguments that are important in chemometrics are as follows.

- Transpose is denoted by ' so that `W'` is the transpose of W.
- Inverse is a function `inv` so that `inv(Q)` is the inverse of a square matrix $\boldsymbol{Q}$.
- The pseudo-inverse can simply be obtained by the function `pinv`, without any further commands, see Figure A.33, where we also verify that the product of a matrix with its pseudo-inverse is the identity matrix.

For a comprehensive list of functions, see the help files that come with Matlab; however, a few that are useful to the reader of this book are as follows. The `size` function gives the dimensions of a matrix, so `size(W)` will return a 2×1 vector with elements, in the example of Section A.5.3.1, of 2 and 3. It is possible to create a new vector, for example, `s = size(W)`, in such a situation `s(1)` will equal 2, or the number of rows. The element `W(s(1), s(2))` represents the last element in the matrix `W`. In addition, it is possible to use the functions `size(W,1)` and `size(W,2)`, which provide the number of rows and columns directly. These functions are very useful when writing simple programs as discussed below.

The `mean` function can be used in various ways. By default, this function produces the mean of each column in a matrix, so that `mean(W)` results in a 1×3 row vector containing the means. It is possible to specify which dimension one wishes to take the mean over, the default being the first one, so `mean(W,2)` is a 2×1 column vector. The overall mean of an entire matrix can be obtained using the mean function twice, that is, `mean(mean(W))`. Note that the mean of a vector is always a single number, whether the vector is a column or row vector. This function is illustrated in Figure A.34. The functions such as `min` and `max` compute minima and maxima of columns, for rows use a transpose, such as `min(X')`.

The functions `std` and `var` calculate the column standard deviations and variances: however, the default is the sample statistics, and in most calculations in chemometrics, we use the population statistics as our aim is to scale the

```
Command Window
>> X = [9 8; 11 4; 5 6];
>> Y = [2 7 1; 5 3 8];
>> Z=[2 3 5; 6 0 1; 11 4 8];
>> X*Y+Z

ans =

    60    90    78
    48    89    44
    51    57    61
```

Figure A.32 Simple matrix operations in Matlab.

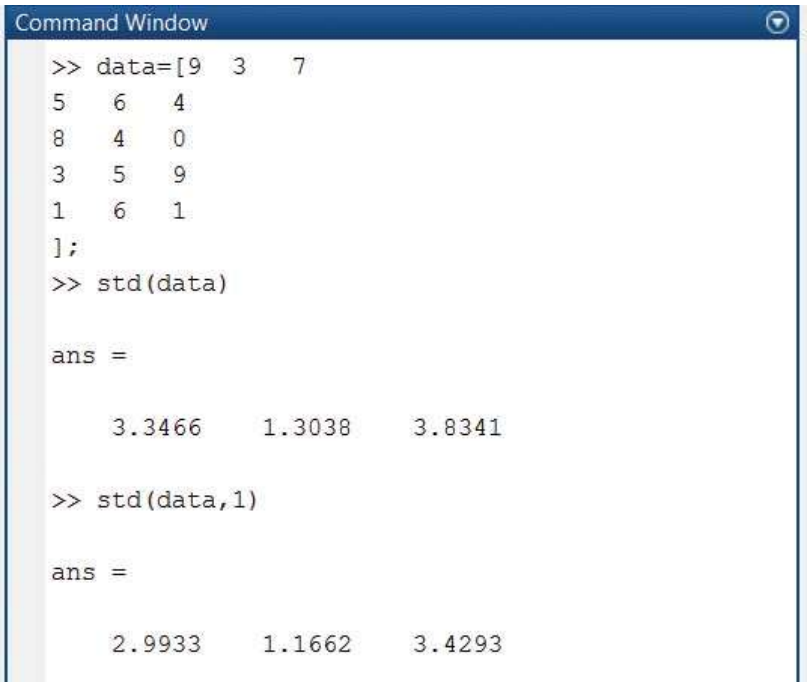


Figure A.35 Calculating standard deviations in Matlab: the second calculation is preferred for most chemometric calculations where the aim is to scale a matrix.

A.5.3.4 Pre-Processing

Pre-processing is slightly awkward in Matlab. One way is to write a small program with loops as described in Section A.5.5. If you think in terms of vectors and matrices, however, it is quite easy to come up with a simple approach. If $\boldsymbol{W}$ is our original 2×3 matrix and we want to mean centre the columns, we can easily obtain a 1×3 vector $\overline{\boldsymbol{w}}$, which corresponds to the means of each column, multiply this by a 2×1 vector $\boldsymbol{1}$ gives a 2×3 vector consisting of the means, and so our new mean centred matrix $\boldsymbol{V}$ can be calculated as $\boldsymbol{V} = \boldsymbol{W} - \boldsymbol{1}\overline{\boldsymbol{w}}$. There is a special function in Matlab called `ones` that creates vectors or matrices that just consist of the number 1, an array `ones (5,3)` would create a matrix of dimensions 5×3 solely of 1s, so a 2×1 vector could be specified using the function `ones(2,1)`. Hence, we can write `V=W-ones(2,1)*mean(W)` to create a new mean centred matrix $\boldsymbol{V}$ as illustrated in Figure A.36.

The experienced user of Matlab can build on this to perform other common methods for pre-processing such as standardisation.

A.5.3.5 Principal Components Analysis

There are several ways of performing PCA in Matlab. We will look at each method and how they relate to the results of NIPALS. We assume $\boldsymbol{X}$ is a $\boldsymbol{I} \times \boldsymbol{J}$ matrix.

SVD is performed by `svd`. A typical statement is `[a b c] = svd(X)`.

- The matrix `b` is of dimensions $\boldsymbol{I} \times \boldsymbol{J}$. The values `b(g,g)` where `g` is the component number consists of the square root of the eigenvalue obtained by NIPALS. All other elements of this matrix are 0. This matrix is always positive.
- The matrix `a` is of dimensions $\boldsymbol{I} \times \boldsymbol{I}$. It corresponds to the scores matrix in NIPALS but is scaled so that the sum of squares of each column is 1, that is, `sum(a(:,g).^2) = 1`. To obtain the same answer as NIPALS, simply multiply each column by its eigenvalue, for example, `b(g,g)*a(:,g)`. Each column represents a component.
- The matrix `c` is of dimensions $\boldsymbol{J} \times \boldsymbol{J}$. This corresponds to the loadings obtained in NIPALS. Each successive column corresponds to a successive component.

There are some important things to note. First of all, the sign after PCA cannot be controlled; thus, in some cases, using `svd` may give components opposite in sign to, for example, Excel. What is important is that the product of scores and loadings always has the same sign. The other is that if the two dimensions are different, only components up to the minimum of $\boldsymbol{I}$ and $\boldsymbol{J}$ are non-zero. The third is that the data are not automatically centred using `svd`, if you want to centre it, you must do it yourself.

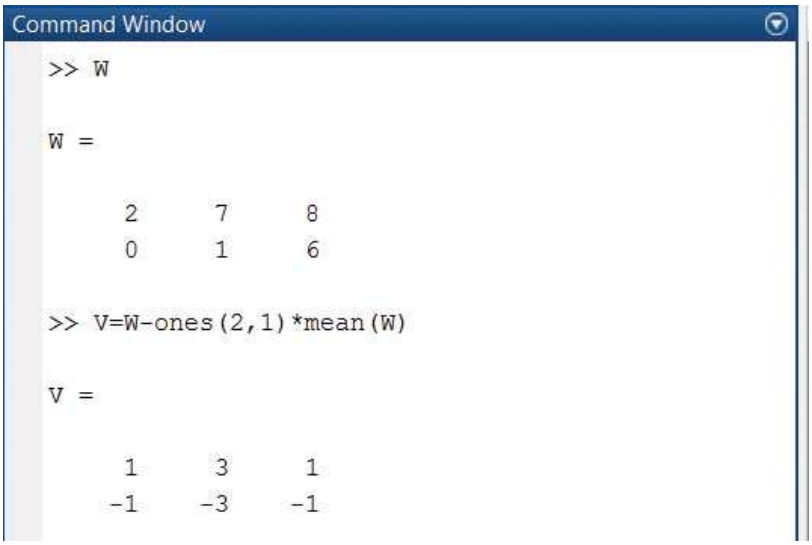


Figure A.36 Mean centring a matrix in Matlab.

The second method is `princomp`. This is due to be withdrawn in later versions of Matlab but is still widely employed. Unlike `svd`, by default the data are centred. Once centred, it is easy to relate the output to that obtained using NIPALS. Assume we calculate `[d e f] = princomp(X)`.

- The $J \times J$ matrix `d` corresponds exactly to the loadings matrix obtained via NIPALS, with each column corresponding to successive components.
- The $I \times I$ matrix `e` corresponds exactly to the scores matrix obtained via NIPALS, with each column corresponding to successive components. Note that using `princomp`, the first matrix is the loadings rather than scores matrix.
- `f` is a column vector rather than a matrix, unlike `svd`, each element corresponding to an eigenvalue. To get to the eigenvalue obtained using NIPALS, multiply each element by $I - 1$.

Remember that always there will be some zero components as in `svd`.

The replacement function `pca` is very similar to `princomp` but has more options. In particular, the calculation can be performed uncentred, so `[p q r] = pca(X,'Centred',false)` will result in the same scores as columns in matrix `q` and loadings as columns in matrix `p` as NIPALS on the raw uncentred data: the eigenvalues have to be multiplied by $I - 1$.

A.5.4 Importing and Exporting Data

In chemometrics, we want to perform operations on numerical data. There are many ways of getting information into Matlab generally straight into matrix format. Some of the simplest are as follows.

- Type the numerical information in the Command window as described above; for small data sets, this is probably easiest.
- If the information is available in a space delimited form with each row on a separate line, for example, as a text file, copy the data, type a command such as

  ```
  X = [
  ```

  but do NOT terminate this by the enter key, then paste the data into the Matlab window and finally terminate with

  ```
  ]
  ```

  using a semicolon if you do not want to see the data displayed again (useful if the original data set is large such as a series of spectra).
- Information can be saved as mat files (Section A.5.2.1) and these can be imported into Matlab. Many public domain chemometrics data sets are stored in this format.

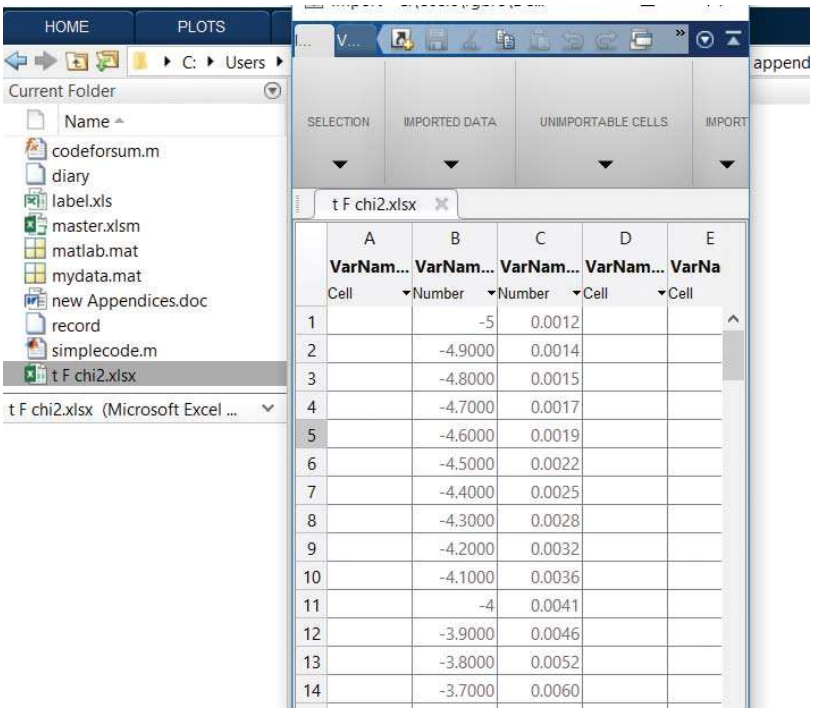


Figure A.37 Importing from Excel to Matlab.

- Information can be imported from Excel. The simplest way is to select a spreadsheet from the current directory. Clicking on this opens it, and one can select the data to import as illustrated in Figure A.37.

Data are best exported as mat files but can also be exported into a variety of formats. The `xlswrite` function is particularly useful when sending data to Excel.

Non-numerical data such as a character data, for example, names, can also be exported.

A.5.5 Introduction to Programming and Structure

For the enthusiasts, it is possible to write quite elaborate programs and develop very professional looking m files. The beginner is also advised to have a basic idea of a few of the main features of Matlab as a programming environment.

First and foremost is the ability to make comments (statements that are not executed), by starting a line with the % sign. Anything after this is simply ignored by Matlab but helps make large m files comprehensible.

Loops commence with the `for` statement, which has a variety of different syntaxes, the simplest being `for i = a : b`, which increments the variable `i` from the number `a` (which must be a scalar) to `b` by 1. An increment (which can be negative and does not need to be an integer) can be specified using the syntax `for i = a : n : b`, notice how, unlike many programming languages, this is the middle value of the three variables. Loops finish with the `end` statement. As an example, the operation of mean centring (Section A.5.3.4) is written in the form of a loop, see Figure A.38, the interested reader should be able to interpret the commands using the information given above. Obviously, for this quite small operation, a loop is not strictly necessary, but for more elaborate programs, it is important to be able to use loops, and there is a lot of flexibility about addressing matrices which make this facility very useful.

`If` and `while` facilities are also useful commands.

Many programmers like to organise their work into functions. In this introductory text, we will not delve too far into this, but a library of m files that consist of different functions can be easily set up as described in Section A.5.2.1.

This text does not aim to be a comprehensive manual about Matlab programming. The best way to learn is from hands-on experience, with a few tips to start up. Every programmer has their own style. If learning for chemometrics, a good starting point is to program in the algorithms of Section A.2, as practice, but also as possible building blocks for more elaborate computations. Most professional chemometrics have some programming experience, the majority in Matlab but also some other environments such as R or Python. Once one environment is mastered, it is normally easy to pick another.

```
Command Window
>> W=[2 7 8 ; 0 1 6];
Y=mean(W);
for i=1:size(W,1);
V(i,:) = W(i,:) - Y;
end
V

V =

     1     3     1
    -1    -3    -1
```

Figure A.38 A simple loop used for mean centring.

A.5.6 Graphics

There are a large number of different types of graph available in Matlab. Below we discuss a few methods that can be used to produce diagrams of the type employed in this text. The enthusiast will soon discover further approaches. Matlab is a very powerful tool for data visualisation.

A.5.6.1 Creating Figures

There are several ways to create new graphs. The simplest is by plotting a command as discussed in the next sections. A new window consisting of a figure is created. Unless otherwise indicated, each time a graphics command is executed, the graph in the figure window is overwritten.

In order to organise the figures better, it is preferable to use the `figure` command. Each time this is typed in the Matlab command window, a new blank figure, as illustrated in Figure A.39, is produced, so typing this three times in succession results in three blank figures each of which is able to contain a graph. The figures are automatically

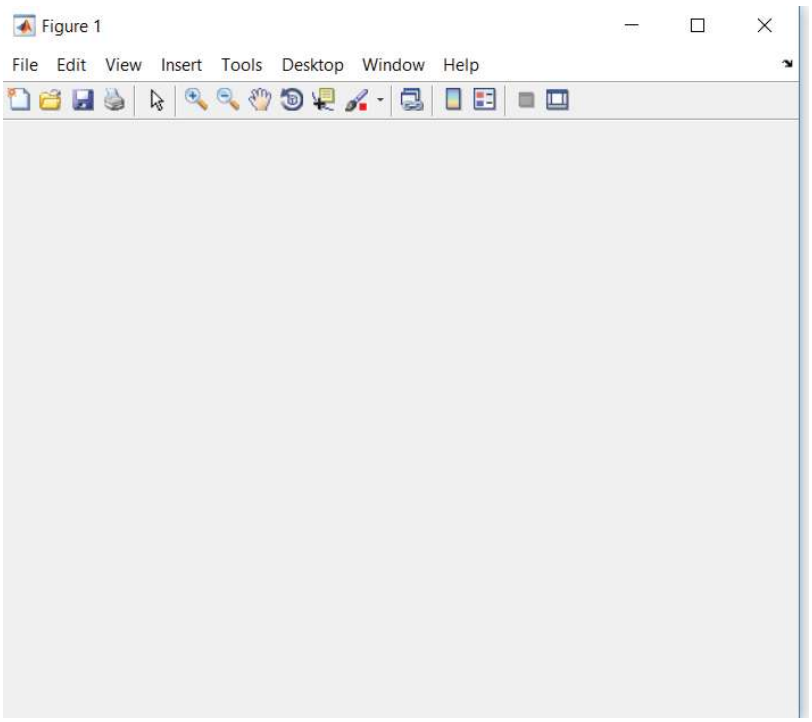


Figure A.39 Blank figure window.

numbered from 1 onwards. In order to return the second figure (number 2), simply type `figure(2)`. All plotting commands apply to the currently open figure. If you wish to produce a graph in the most recently opened window, it is not necessary to specify a number. Therefore, if you were to type the command `figure` three times, unless specified, the current graph will be displayed in Figure 3. The figures can be accessed either as small icons or through the `Window` menu item. It is possible to skip figure numbers; thus, the command `figure(10)` will create a figure number 10, even if no other figures have been created.

Each time you plot a graph if you do not open a new figure, the new graph will overwrite the old one. So if you want `figure; plot(x(:,1),x(:,2)); plot (y(:,1), y(:2))` where a semicolon is used to separate commands, you will only see the graph of `y(:,2)` versus `y(:,1)`. To avoid this, use the very useful `hold on` facility. This allows the user to superimpose several graphs. The statements `figure; plot(x(:,1),x(:,2)); hold on ; plot (y(:,1), y(:2))` will plot both on the same figure. This is illustrated in Figure A.40.

If you want to produce several small graphs on one figure, use the `subplot` command. This has the syntax `subplot(n,m,i)`. It divides the figure into $n \times m$ small graphs and puts the current plot into the ith position, where the first row is numbered from 1 to m, the second from $m+1$ to $2m$ and so on. Figure A.41 illustrates the case where the commands `subplot(2,2,1)` and `subplot(2,2,2)` have been used to divide the window into a 2×2 grid, capable of holding up to four graphs, and figures have been inserted into positions 1 (top left) and 2 (top right). Further figures can be inserted into the grid in the vacant positions, or the current figures can be replaced and overwritten.

Once the figure is complete, you can copy it using the `Copy Figure` menu item and then place it in documents. In this section, we will illustrate the figures by screen snapshots showing the grey background of the Matlab screen. Alternatively, the figures can be saved in Matlab format, using the menu item under the current directory, as a `fig` file, which can then be opened and edited in Matlab in the future.

If you have several figures and want to remove them from Matlab (they may have been saved or the program may generate more figures than you need), the statement `close all` starts from afresh. Using just `close` only removes the most current figure.

A.5.6.2 Line Graphs

The simplest type of graph is a line graph. If $\boldsymbol{Y}$ is a vector, then `plot(Y)` will simply produce a graph of each element against row number. Often, we want to plot a row or column of a matrix against element number, for example, if each successive point corresponds to a point in time or a spectral wavelength. This is quite is to do, the command `plot(X(:,2))` plots the second column of $\boldsymbol{X}$. Plotting a subset is also possible, for example, `plot(X(11:20,2))` produces a graph of column 2 rows 11–20, in practice allowing an expansion of the interesting region. Plotting more than one row against another can be done by specifying two arguments; hence, for example, `plot(X(:,1),X(:,2))` plots the first column on the horizontal scale and the second column on the vertical scale.

Once you have produced a line graph, it is possible to change its appearance. There are numerous ways of doing this, which can be learnt by experience, but there are two approaches.

The first approach is to expand the `plot` statement using specifiers. If writing a program to be used regularly so that plots of a certain type are always generated, this is the preferred approach. Common reasons are to change the line colour or whether there are markers (symbols). The Matlab help files provide a comprehensive list, but this can be specified using quotes, within the plot command. Common colours include blue ('b'), red ('r'), green ('g') and black ('b'). Common markers include circles ('o'), squares ('s') and diamonds ('d'). The line type can be continuous ('–'), none – just markers ('·'), or dotted (':'). These specifiers are combined together. For example, `plot(x(:,1),x(:,2),' ro')` will plot a graph with red circular markers and no line, as illustrated in Figure A.42.

Almost everything you want can be specified within a program, such as axis legends, chart titles and limits to axes, which the reader will learn by experience. One very useful statement is the `axis square` statement. This makes the axes square. So long as the axes are scaled correctly, this ensures that the angles in the graph reflect the angles in the original data space. Figure A.43 illustrates the importance. In the left-hand graph, the two lines do not appear to be at right angles, whereas in the right-hand graph they are. In certain applications, it is useful to see the true angles between vectors.

The second approach is interactive. Rather than type in statements, it is possible to change properties interactively. On the figure, click the white arrow on the top menu, and then any part of the graph you like. This will bring up the property editor as illustrated in Figure A.44. This is very flexible and allows the appearance of graphs to be customised. Particularly useful is to be able to label the axes and change the axis limits.

Almost all specifications can be changed both using the Property Editor and within code, and sometimes a combination is quickest.

In addition to line graphs obtained using the `plot` statement, there are several other common types of graphs. For brevity, we do not describe these, except for 3D graphics in Section A.5.6.4.

A.5.6.3 Labelling Points

Points in a graph can be labelled using the `text` command. The basic syntax is `text (A,B,name)`, where the `A` and `B` are arrays with the same number of elements, and it is recommended that `name` is an array of names or characters likewise with the identical number of elements. Matlab is rather awkward in handling of string (or character) variables. There are various ways telling Matlab that a variable is a string rather than numerical variable. Any data surrounding by single quotes is treated as a string, so the array `c = ['a' ; 'b' ; 'c']` will be treated by Matlab as a 3×1 character array. Figure A.45 illustrates the use of this method. Note that in order to prevent the labels overlapping with the points in the graph, leaving one or two spaces before the actual text helps. It is possible to move the labels, in the interactive graph editor if there is still some overlap, or else code this in. The appearance of the labelling can be altered by adding a specification; for example, `text(x(1,:),x(2,:),c,'Color','b','Fontsize',14)` specifies that the text is blue and with a font size of 14. The effect is illustrated in Figure A.45. The positions of the labels can be changed manually, or in order to prevent overlap with axes, the axis limits can also be changed.

Sometimes, the labels are originally in a numerical format; for example, they may consist of numerical values of points in time or wavelengths. For Matlab to recognise this, the numbers can be converted to strings using the `num2str` function, so that the numbers are changed to text and can be used for labelling points, for example, important wavelengths or key samples.

A.5.6.4 Three-Dimensional Graphics

One of the most flexible and important aspects of Matlab are the comprehensive 3D graphics facilities. We have used these in Chapter 7 when we illustrate the 3D PC plots. Matlab has facilities for contour plots, mesh plots and so on, but in this section, we focus only on the 3D plot statement.

Consider a scores matrix of dimensions 25×3 ($\boldsymbol{T}$) and a loadings matrix of dimensions 12×3 ($\boldsymbol{P}$); note that the default in Matlab is to represent the loadings as column rather than row vectors, see the discussion about PCA – whereas in this book, we normally represent loadings as row vectors. The statement `plot3(T(:,1),T(:,2),T(:,3))` produces a graph of all the three columns of the scores against one another, see Figure A.46. Often, the default orientation is not the most informative for our purposes, and we may well wish to change this. There are a huge number of commands in Matlab to do this, which is a big bonus for the enthusiast, but for the first-time user, the easiest is to select the rotation icon, and interactively change the view (see Figure A.47). If that is the desired view, leave go of the icon.

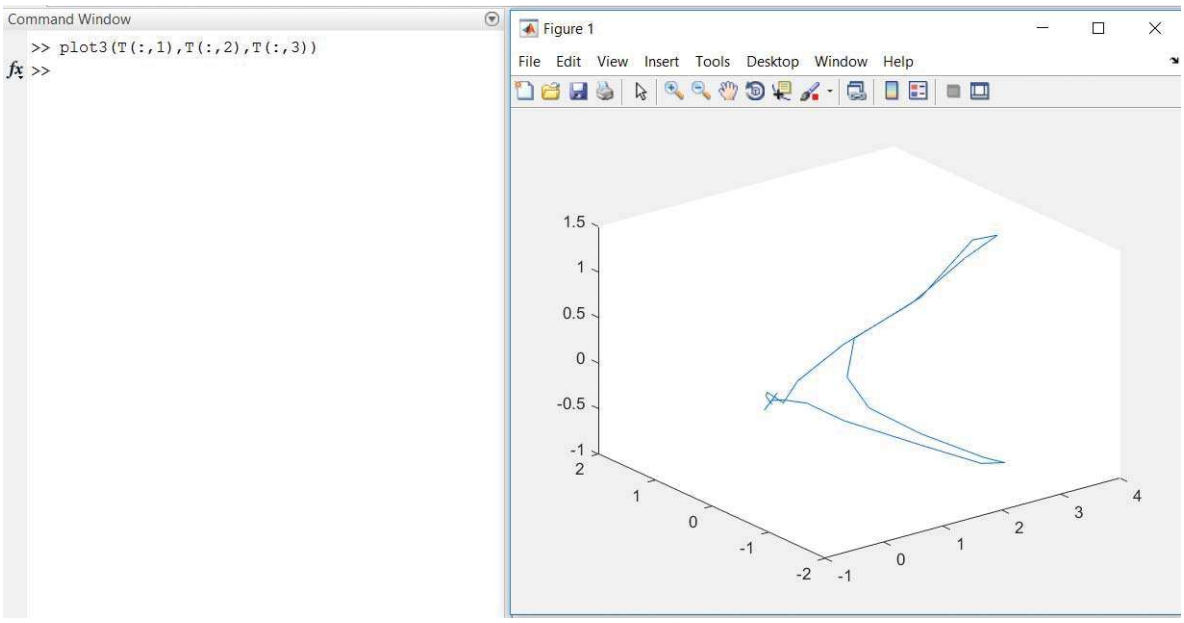


Figure A.46 A 3D scores plot.